
1. Demand Paging — Complete Understanding (Expanded Edition)

Step 0: The Real Problem (Why This Exists)

The physical reality of every computer:

Component	Typical Size	Speed	What It Actually Means
CPU Registers	~1 KB total	~1 nanosecond	Holds exactly what the CPU is doing <i>right now</i>
RAM	4–64 GB	~100 nanoseconds	Holds what the program is actively using
Disk (SSD/HDD)	256 GB – 8 TB+	~100,000 nanoseconds (SSD) to 10,000,000 ns (HDD)	Holds everything else

The conflict: Your program might be 10 GB. Your RAM might be 8 GB. You cannot fit the whole program in RAM at once.

The question: How do we run a 10 GB program in 8 GB of RAM?

The answer (before demand paging): We couldn't. Programs had to be smaller than RAM.

The answer (with demand paging): We pretend we have infinite RAM by keeping most of the program on disk and only bringing pieces into real RAM when actually needed.

Step 1: Memory Hierarchy — Why It Works

The hierarchy exists because of a fundamental trade-off:

| *Fast memory is expensive. Cheap memory is slow.*

We build a pyramid:

- **Top:** Tiny, blazing fast, very expensive (registers, cache)
- **Middle:** Moderate size, fast, moderately expensive (RAM)
- **Bottom:** Huge, slow, cheap (disk)

The trick: Most of the time, you only use a small portion of your program. If we can keep *that small portion* in fast memory and the rest in slow memory, we get the speed of fast memory with the capacity of slow memory.

Step 2: From Swapping to Demand Paging

OLD METHOD: Swapping (Pre-Demand Paging)

What happens:

1. You double-click a program
2. The ENTIRE program is copied from disk to RAM
3. Execution starts
4. If RAM is full, the ENTIRE program of another process is copied out to disk (swap space)

Why it's bad:

- If your program is 2 GB but you only use 50 MB of it in the first hour, you wasted 1.95 GB of RAM
- Startup is slow because you wait for the full copy
- Swapping entire processes is brutal — everything stops

Analogy: You go to the library to research one chapter. You check out ALL 50 books in the series. Your desk is buried. You can't work on anything else.

NEW METHOD: Demand Paging

What happens:

1. You double-click a program
2. Only the parts absolutely required to *start* are loaded (often just 1-2 pages)
3. The rest stays on disk
4. When the CPU needs a new part it's brought in on demand
5. If RAM gets full, only the *least useful* pages are removed (not whole programs)

Analogy: You go to the library. You place ONE book on your desk. When you need Chapter 5, you fetch only Chapter 5. When you need Chapter 12, you fetch only Chapter 12. If your desk is full, you return the chapter you haven't looked at in 2 hours.

Step 3: What Exactly Is a “Page”?

Formal Definition

A Page is a fixed-size contiguous block of virtual memory, typically 4 KB (4096 bytes), that is the fundamental unit of transfer between disk and RAM.

Why 4 KB specifically?

Size	Why It Matters
4 KB = 4096 bytes = 2^{12} bytes	Powers of 2 make address calculation trivial for hardware
Small enough	Fine-grained — you don't waste much RAM loading unused stuff
Large enough	Amortizes disk access cost — reading 4 KB from disk takes almost the same time as reading 1 byte

What does “fixed-size” actually mean?

Every page in the system is exactly the same size. If your program is 10,000 bytes and pages are 4 KB:

- Page 0: bytes 0–4095
- Page 1: bytes 4096–8191
- Page 2: bytes 8192–10000 (plus some unused space up to 12287)

The unused space in the last page is called **internal fragmentation** — it's a small waste we accept for the massive simplification fixed sizes provide.

Step 4: How Demand Paging Works (Full Step-by-Step)

The Players

Player	Role
CPU	Generates memory addresses, executes instructions
MMU (Memory Management Unit)	Hardware chip that translates virtual physical addresses
Page Table	Data structure in RAM that maps virtual pages physical frames or disk locations
OS	Handles page faults, decides what to load/evict
Disk	Holds pages not currently in RAM

Step 1: Program Starts

What actually happens:

1. The OS creates a process
2. It sets up a Page Table for this process
3. It loads **only the first page** (or pages containing the entry point) into RAM
4. **All other pages are marked "Not Present"** in the Page Table

What the Page Table looks like initially:

Virtual Page	Present?	Physical Frame	Disk Location
0	Yes	Frame 12	—
1	No	—	Disk block 4,082
2	No	—	Disk block 4,083
3	No	—	Disk block 4,084
...	No	—	...

Step 2: CPU Requests Data

The CPU doesn't know about pages. It just generates a **virtual address**.

Example:

- Virtual Address = 0x00005000 (hexadecimal)

- Page size = 4 KB = 0x1000
- **Page number** = $0x00005000 \div 0x1000$ = Page 5
- **Offset within page** = $0x00005000 \% 0x1000$ = 0x0000

So the CPU wants **Page 5, offset 0**.

Step 3: Check Page Table (The MMU Does This)

The MMU hardware automatically:

1. Splits the virtual address into (Page Number, Offset)
2. Looks up Page 5 in the Page Table
3. Checks the “Present” bit

Case 1: Page IS in RAM (Page Hit)

Virtual Page	Present?	Physical Frame
5	Yes	Frame 23

What happens:

1. MMU sees Present = Yes
2. MMU knows Page 5 is in Physical Frame 23
3. Physical Address = (Frame 23 × 4096) + offset 0 = Frame 23, byte 0
4. RAM returns the data
5. **Total time:** ~100 nanoseconds (RAM speed)

No OS involved. No interrupt. Pure hardware speed.

Case 2: Page NOT in RAM (Page Fault)

Virtual Page	Present?	Physical Frame	Disk Location
5	No	—	Disk block 8,291

What happens — full sequence:

Step	Who	What Happens	Time
1	MMU	Sees Present = No, triggers Page Fault Exception	Immediate

2	CPU	Saves current state, switches to kernel mode	~tens of ns
3	OS	Runs Page Fault Handler	—
4	OS	Checks: Is this a valid page? (Security check — is program allowed to access page 5?)	~hundreds of ns
5	OS	If invalid Segmentation Fault (kill program)	—
6	OS	If valid find a free frame in RAM	~microseconds
7	OS	If no free frame RUN PAGE REPLACEMENT ALGORITHM (pick victim)	~microseconds
8	OS	If victim is dirty write it to disk first	~milliseconds
9	OS	Read requested page from disk into the freed frame	~5–10 milliseconds
10	OS	Update Page Table: mark Page 5 as Present, set Physical Frame	~hundreds of ns
11	OS	Restart the instruction that caused the fault	—
12	CPU	Re-executes the same instruction now it's a Page Hit	~100 ns

Total time for one page fault: ~5–15 milliseconds

Critical insight: A page fault is ~100,000× slower than a RAM access. One page fault = you could have done 100,000 RAM accesses in the same time.

Step 5: Page Fault — Deep Dive

What is a Page Fault, Really?

A Page Fault is a hardware exception (trap) raised by the MMU when the CPU attempts to access a virtual page that is not currently mapped to a physical frame in RAM.

It is **not an error** in the traditional sense. It's a normal, expected event in demand paging systems. The OS handles it and execution continues.

Types of Page Faults

Type	Cause	Outcome
Major Page Fault	Page not in RAM, must read from disk	Slow (~5-10 ms)
Minor Page Fault	Page is in RAM but not in this process's page table (shared memory)	Fast (~microseconds)
Invalid Page Fault	Program accessed memory it doesn't own	Segmentation fault, program killed

Step 6: What If RAM Is Full? — Page Replacement

The Problem

RAM has, say, 4 GB = ~1,048,576 frames (at 4 KB/page).

All frames are occupied. A new page fault occurs. Where do we put the new page?

The Solution: Evict Someone

Eviction means:

1. Pick a page currently in RAM
2. Remove it from RAM
3. Put the new page in that frame
4. Update Page Tables for both the evicted and new pages

The Question: Which Page to Evict?

This is the **Page Replacement Problem**. The algorithm that decides is called a **Page Replacement Algorithm**.

Page Replacement Algorithms (Brief Overview)

Algorithm	What It Does	Simple Analogy
-----------	--------------	----------------

FIFO (First-In-First-Out)	Evict the page that has been in RAM longest	Throw out the book you've had on your desk the longest
LRU (Least Recently Used)	Evict the page unused for the longest time	Throw out the book you haven't opened in the longest time
Optimal	Evict the page that won't be used for the longest time in the future	Throw out the book you won't need until next month (theoretical only — needs future knowledge)
Clock (Second Chance)	Approximation of LRU using a "used" bit	Give books a second chance if they've been referenced recently

Important: No algorithm is perfect. All involve trade-offs between implementation complexity and hit rate.

Step 7: Clean vs Dirty Pages (Critical Missing Concept)

When we evict a page, we must decide: **Do we need to save it to disk first?**

Clean Page

A Clean Page is a page in RAM that has NOT been modified since it was loaded from disk.

What this means:

- The disk already has an exact copy
- The RAM copy and disk copy are identical
- **Action on eviction:** Just delete it from RAM. No disk write needed.

How the system knows: The MMU has a **Dirty Bit** in the Page Table entry. If Dirty = 0, the page is clean.

Dirty Page

A Dirty Page is a page in RAM that HAS been modified by the CPU (written to).

What this means:

- The RAM copy is newer than the disk copy
- If we just delete it from RAM, we **lose data**

- **Action on eviction:** Must write the page back to disk first, THEN free the frame

How the system knows: Dirty Bit = 1 in the Page Table.

Why This Matters

Scenario	Time Penalty
Evict clean page	~1 microsecond (just update tables)
Evict dirty page	~5–10 milliseconds (write to disk) + ~5–10 milliseconds (read new page)

Double page fault time if dirty!

Systems try to minimize dirty pages or write them back early (background writing).

Step 8: Formal Definition (Improved)

Demand Paging is a virtual memory management technique in which pages are loaded from secondary storage (disk) into primary storage (RAM) only at the moment they are referenced by a running process, rather than being loaded entirely in advance.

Key phrases explained:

- “Referenced by a running process” = the CPU actually tries to read/write that address
 - “Rather than loaded entirely in advance” = contrast with old swapping systems
 - “Secondary storage” = disk/SSD
 - “Primary storage” = RAM
-

Step 9: Advantages & Disadvantages (Expanded)

Advantages

Advantage	What It Actually Means
Efficient RAM usage	RAM contains only actively used pages, not idle code/data

Faster program startup	Only 1-2 pages loaded initially, not the whole program
Larger address space	Programs can be bigger than physical RAM (virtual memory)
Supports more processes	10 programs of 2 GB each can run in 8 GB RAM if they don't all use everything at once
Simplifies programming	Programmers don't worry about physical RAM limits

Disadvantages

Disadvantage	What It Actually Means
Page faults are expensive	Each fault = disk I/O = CPU sits idle for milliseconds
Thrashing	If system is overloaded, it spends more time paging than executing system becomes unusable
Complex hardware required	Needs MMU, page tables, TLB, dirty bits — adds cost and complexity
Unpredictable performance	Same program might run fast (few faults) or slow (many faults) depending on what else is running

Step 10: Real System Implementation

Where Demand Paging Is Used

System	How It Uses Demand Paging
Linux	Every process uses demand paging. The <code>mmap()</code> system call explicitly maps files with on-demand loading. Kernel uses <code>swap</code> partition/pagefile for eviction
Windows	Memory Manager uses “demand paging with clustering” — brings in requested page plus a few adjacent pages (prefetching)
Android/iOS	Mobile devices with limited RAM rely heavily on demand paging + aggressive app killing (Low Memory Killer)

macOS	Unified memory architecture uses demand paging across CPU/GPU shared memory
-------	---

Modern Optimizations

Technique	What It Does	Benefit
TLB (Translation Lookaside Buffer)	Hardware cache of recent virtual physical translations	Avoids page table walk for repeated accesses
Prefetching	OS predicts which pages you'll need next and loads them early	Reduces future page faults
Huge Pages	Use 2 MB or 1 GB pages instead of 4 KB	Reduces TLB misses, better for large datasets
Swap Compression (zswap/zram)	Compress pages in RAM instead of writing to disk	Faster than disk I/O

Step 11: Full Worked Example (With All Steps Shown)

Given:

- RAM can hold **2 pages** (tiny for illustration)
- Program accesses pages in this exact order: **A B C A**

Page Table & RAM State Tracking

We'll track:

- **RAM contents** (which pages are loaded)
- **Page Table** (Present/Not Present for each page)
- **Page Fault count**
- **What action was taken**

Access 1: Page A

Step	Action
------	--------

CPU requests	Page A
Page Table check	A: Not Present
Result	PAGE FAULT
RAM state before	[empty]
Action	Load A into Frame 0
RAM state after	[A]
Page Table update	A: Present, Frame 0
Page Faults so far	1

Access 2: Page B

Step	Action
CPU requests	Page B
Page Table check	B: Not Present
Result	PAGE FAULT
RAM state before	[A]
Action	Load B into Frame 1
RAM state after	[A, B]
Page Table update	B: Present, Frame 1
Page Faults so far	2

Access 3: Page C

Step	Action
CPU requests	Page C
Page Table check	C: Not Present
Result	PAGE FAULT

RAM state before	[A, B] — FULL
Action	Must evict someone. Using FIFO: A was first in, so evict A
	Check: Is A dirty? Assume NO (clean) just remove
	Load C into Frame 0 (A's old spot)
RAM state after	[B, C]
Page Table update	A: Not Present; C: Present, Frame 0
Page Faults so far	3

Access 4: Page A (again)

Step	Action
CPU requests	Page A
Page Table check	A: Not Present (we evicted it!)
Result	PAGE FAULT
RAM state before	[B, C] — FULL
Action	Evict B (FIFO: B was first in among current pages)
	Check: Is B dirty? Assume NO just remove
	Load A into Frame 1 (B's old spot)
RAM state after	[C, A]
Page Table update	B: Not Present; A: Present, Frame 1
Page Faults so far	4

Final Summary

Access	Page	RAM Before	Action	RAM After	Fault?
1	A	[]	Load A	[A]	Yes
2	B	[A]	Load B	[A, B]	Yes

3	C	[A, B]	Evict A, Load C	[B, C]	Yes
4	A	[B, C]	Evict B, Load A	[C, A]	Yes

Total Page Faults: 4 out of 4 accesses

Hit Rate: 0% — terrible because the working set (3 pages) exceeds RAM capacity (2 pages). This is approaching **thrashing**.

What If RAM Held 3 Pages?

Access	Page	RAM	Fault?
1	A	[A]	Yes
2	B	[A, B]	Yes
3	C	[A, B, C]	Yes
4	A	[A, B, C]	NO — A is still there!

Total Faults: 3 (vs 4 before). **Hit Rate: 25%**

This demonstrates the **working set model**: if RAM holds all pages a program is actively using, faults drop dramatically.

Step 12: Key Definitions for Your Notebook

Term	Definition
Page	Fixed-size block of virtual memory (typically 4 KB), the unit of transfer between disk and RAM
Frame	A page-sized slot in physical RAM
Page Table	Per-process data structure mapping virtual page numbers to physical frame numbers or disk locations
Page Fault	Hardware exception when CPU accesses a page not present in RAM

Demand Paging	Memory management technique loading pages only when referenced, not in advance
Page Replacement	Algorithm to select which page to evict when RAM is full
Dirty Bit	Flag in page table indicating whether a page has been modified since loading
Clean Page	Unmodified page — can be evicted without disk write
Dirty Page	Modified page — must be written to disk before eviction
Thrashing	System state where excessive page faults consume most CPU time
Working Set	Set of pages a process actively uses in a given time window
TLB	Hardware cache speeding up virtual-to-physical address translation
Virtual Memory	Abstraction giving each process the illusion of a large, private, contiguous address space

Step 13: Practice Problems (Full Solutions)

Problem 1 (Basic)

If a program accesses a page not in RAM, what happens?

Complete Solution:

1. The CPU generates a virtual address for the desired data
2. The MMU splits this into (Page Number, Offset)
3. The MMU checks the Page Table entry for that page number
4. The Present bit is 0 (Not Present)
5. The MMU triggers a **Page Fault exception**
6. The CPU switches to kernel mode and jumps to the OS Page Fault Handler
7. The OS validates the access (security check — is this page part of this process?)
8. If invalid: send SIGSEGV (segmentation fault), terminate program
9. If valid: OS finds a free frame (or evicts one using replacement algorithm)
10. OS issues disk I/O to read the page from its disk location
11. Disk controller reads the page into the frame

12. OS updates Page Table: set Present=1, record Physical Frame number
 13. OS restarts the faulting instruction
 14. CPU re-executes the instruction — this time it's a hit
 15. Execution continues normally
-

Problem 2 (Conceptual)

Why is demand paging better than loading everything?

Complete Solution:

Aspect	Load Everything	Demand Paging
RAM usage	Wasted — unused code/data sits in RAM	Efficient — only active pages use RAM
Startup time	Slow — must read entire program from disk	Fast — read 1-2 pages, start immediately
Multitasking	Limited — each program hogs RAM	Better — many programs share RAM via partial loading
Program size limit	Hard cap at physical RAM size	Virtual memory allows programs >> RAM
I/O overhead	Massive upfront read	Spread out, only for actually used pages

Example with numbers:

- Program size: 100 MB
 - Typical usage in first 5 minutes: 5 MB of code + 2 MB of data = 7 MB
 - **Load everything:** 100 MB read from disk, 100 MB RAM occupied
 - **Demand paging:** ~7 MB read from disk, ~7 MB RAM occupied
 - **Savings:** 93% less disk I/O, 93% less RAM used
-

Problem 3 (Thinking)

Why are too many page faults bad?

Complete Solution:

Time comparison (typical values):

Operation	Time
RAM access	~100 nanoseconds = 0.0001 milliseconds
Disk I/O (one page)	~10 milliseconds = 10,000,000 nanoseconds
Ratio	Disk is ~100,000× slower than RAM

Scenario:

- Program needs 1,000,000 memory accesses
- Ideal (all in RAM): $1,000,000 \times 0.0001 \text{ ms} = \mathbf{100 \text{ ms}}$ (0.1 seconds)
- With 10% page fault rate: 900,000 RAM hits + 100,000 faults
 - RAM hits: $900,000 \times 0.0001 \text{ ms} = 90 \text{ ms}$
 - Faults: $100,000 \times 10 \text{ ms} = \mathbf{1,000,000 \text{ ms} = 1,000 \text{ seconds} \approx 16.7 \text{ minutes}}$
 - **Total: ~16.7 minutes vs 0.1 seconds = 10,000× slower**

This is thrashing. The CPU spends 99.99% of its time waiting for disk. The system appears frozen.

Real-world indicator: Your computer is “swapping to death” — LED solid on, mouse jerky, everything slow. That’s thrashing.

Final Intuition (Locked In)

Demand Paging = “Lazy Loading of Memory”

Analogy	Technical Reality
You don’t prepare everything in advance	OS doesn’t load all pages at startup
You respond only when needed	OS loads page only on CPU reference
If your desk is full, you return the least useful book	If RAM is full, evict least useful page
If you wrote notes in a book, you must save them before returning	If page is dirty, write to disk before eviction

2. Why Demand Paging Matters (Big Picture — Deep Understanding)

Step 0: What This Section Is Actually About

This section answers one question from three angles:

Angle	The Question
Capability	How can programs larger than RAM even run?
Speed	Why don't we wait 5 minutes every time we open an app?
Efficiency	How does the system run 50 apps in 8 GB RAM without dying?

Without demand paging, modern computing as you know it would not exist. This is not an exaggeration.

1. The Illusion of Infinite Memory

The Mathematical Impossibility

Given:

- Your game: 100 GB
- Your RAM: 16 GB
- Required: Load entire game into RAM to run

Problem: $100\text{ GB} > 16\text{ GB}$. Impossible. Game cannot run.

But it does run. Here's exactly how.

Virtual Memory + Demand Paging = The Solution

What Virtual Memory does:

- Gives every program its own private address space (e.g., 64-bit systems = 16 exabytes theoretical address space)
- This address space is **virtual** — not backed by physical RAM
- The program thinks it has infinite memory
- The OS maps only the “active” parts to real RAM

What Demand Paging does:

- Only maps (loads into RAM) the pages the CPU actually touches
- Everything else stays on disk, invisible to the program but perfectly accessible when needed

The program never knows. It asks for address 0x00000000 through 0x174848BFF (100 GB worth). The OS says “sure, here you go” — but secretly only 200 MB of that is in RAM right now.

The Working Set (Formal Definition)

The Working Set is the set of virtual memory pages that a process has referenced within a specific time window and that are currently resident in physical RAM.

Breaking this down:

Term	Meaning
“Set of virtual memory pages”	A collection of page numbers (like Page 5, Page 12, Page 203)
“Referenced within a specific time window”	The CPU actually read from or wrote to these pages recently (last T milliseconds)
“Currently resident in physical RAM”	They are actually loaded and present right now

Example — Gaming Scenario:

Game Component	Size	In Working Set?	Why
Current level geometry	800 MB	Yes	CPU rendering it right now
Player character model	50 MB	Yes	Being animated

Current soundtrack	30 MB	Yes	Playing now
Next level (3 areas away)	900 MB	No	Not needed yet
Title screen assets	200 MB	No	Not displayed
All 47 language packs	4 GB	No	Only English selected
Full video cutscenes	8 GB	No	Not triggered yet

Working Set: ~880 MB (active gameplay assets)

Total Program: 100 GB

RAM Used: ~880 MB + OS overhead $\approx$ 1.5 GB

Result: Game runs smoothly in 16 GB RAM with 14.5 GB to spare.

Why This Works: Locality of Reference

Demand paging is not magic. It exploits a real, measurable property of how programs behave.

Temporal Locality

Temporal Locality: If a memory location is accessed, it is likely to be accessed again soon.

What this means literally:

- You just loaded your inventory screen → you'll probably access inventory data again in 30 seconds
- You just jumped → the jump physics code will run again when you jump next
- You just saved the game → the save system code might run again

Technical implication:

- Once a page is loaded for the first time, it stays in RAM
- The next access is a **hit** (fast RAM access, ~100 ns)
- No second page fault for that page

Example with numbers:

- Page 5 (inventory code): First access = 10 ms (fault + disk read)
- Page 5, second access 5 seconds later: 0.0001 ms (RAM hit)

- **Speedup: 100,000×**
-

Spatial Locality

Spatial Locality: If a memory location is accessed, nearby locations are likely to be accessed soon.

What this means literally:

- You accessed byte 1000 in an array you'll probably access byte 1004 next
- You loaded the texture for a character's head you'll need the texture for the body next
- You started executing a function at address X the next instruction is at address X+4

Technical implication:

- Pages contain contiguous memory (4 KB blocks)
- Accessing one byte in a page often means accessing other bytes in the same page
- Loading the whole page at once (4 KB) catches the nearby data efficiently

Example:

- Array `int scores[1000]` occupies 4,000 bytes = roughly 1 page
 - You access `scores[0]` page fault, load entire page
 - Next 999 accesses (`scores[1]` through `scores[999]`) are all in that same page
 - **1 fault serves 1000 accesses**
-

Why Locality Is Essential

Scenario	Locality?	Page Fault Rate	System State
Normal program execution	Strong	0.1% – 1%	Healthy
Random memory access pattern	None	50%+	Dying
Sequential array scan	Very strong	Near 0%	Excellent
Jumping across all 100 GB randomly	None	100%	Thrashing

Without locality, demand paging collapses. Every access would fault. The system would spend 100% of its time reading from disk.

2. Faster Startup

The Old Way (No Demand Paging)

Scenario: Opening a web browser

Step	Action	Time
1	Read all 150 MB of browser code from disk	~1.5 seconds
2	Read all 200 MB of default plugins	~2 seconds
3	Read all 50 MB of default settings/data	~0.5 seconds
4	Read all 500 MB of cached history/bookmarks	~5 seconds
5	Finally start executing first instruction	~9 seconds total

Problem: You don't need all 900 MB to show a blank window with an address bar. But the system loaded it all anyway.

The Demand Paging Way

Step	Action	Time
1	Read 2 MB (kernel + initial code) from disk	~20 ms
2	Start executing — window appears	~20 ms
3	Load 5 MB more as you type in address bar	~50 ms
4	Load 20 MB more as page renders	~200 ms
5	Other 873 MB stays on disk until/if needed	0 ms

Apparent startup time: ~20 ms (feels instant)

Actual total loaded after 5 minutes of browsing: ~50 MB

Savings: 850 MB of unnecessary disk I/O avoided.

What “Execution Begins Immediately” Actually Means

Technical sequence:

1. OS creates process, allocates page table (all entries = “Not Present”)
2. OS loads **exactly 1 page**: the page containing the program’s entry point (first instruction)
3. CPU starts executing
4. First instruction runs → probably calls another function
5. That function is in a different page → **page fault** → load that page
6. Continue execution
7. Repeat as needed

The program is “running” after step 2. It hasn’t loaded 99% of itself, but it’s genuinely executing instructions.

Advanced: Prefetching

Modern systems don’t wait for the fault. They predict:

Prefetching: The OS or hardware predicts which pages will be needed next and loads them before the CPU asks.

Example:

- CPU accesses Page 5
- System notices pattern: historically, after Page 5 comes Page 6, then Page 7
- System silently loads Page 6 and Page 7 in background
- When CPU asks for Page 6: **already in RAM** → no fault

Prefetching + Demand Paging:

- Demand paging handles the unexpected (“I need this NOW”)
 - Prefetching handles the predictable (“I bet you’ll need this soon”)
 - Together: minimal faults, maximum speed
-

3. Efficiency — Memory Optimization

The Problem: Programs Are Wasteful

A typical large program contains:

Section	When It’s Used	Size
---------	----------------	------

Main game loop	Every frame	20% of code
Rendering engine	Every frame	25% of code
Physics engine	Every frame	15% of code
Audio system	Every frame	10% of code
Error handling	Only when bugs occur	5% of code
Debug logging	Only in dev mode	10% of code
Update checker	Once per day	2% of code
First-run tutorial	Once ever	5% of code
Uninstaller	Never while running	3% of code
47 localization files	Only 1 language used	15% of data

Total code/data: 100%

Used in a normal gaming session: ~70%

Used in any given 10-second window: ~15%

Without Demand Paging

All 100% loaded into RAM permanently.

Outcome	Result
30% of RAM holds dead code	Waste
Less room for other programs	Reduced multitasking
More disk I/O at startup	Slower boot
RAM pressure increases	More swapping, more faults

With Demand Paging

Outcome	Result
Only ~15% in RAM at any moment	Efficient

85% stays on disk, costs nothing	Free
Other programs get that RAM	Better multitasking
Rarely-used code never loads	Zero cost for dead code

Hot Data vs Cold Data

Term	Definition	Where It Lives
Hot Data	Frequently accessed, recently used	RAM (fast)
Warm Data	Accessed occasionally	RAM or disk (migrates)
Cold Data	Rarely or never accessed	Disk (slow, cheap)

The system automatically manages this migration:

- Hot stays in RAM (protected from eviction)
 - Warm might get evicted, but faulted back quickly if needed
 - Cold never leaves disk, never wastes RAM
-

Step 4: The Nightmare Scenario — What If We Had No Demand Paging?

System Designer's Dilemma

You are building an OS. You have:

- 8 GB RAM
- Users want to run: Browser (1 GB), Game (4 GB), IDE (2 GB), Music player (500 MB), Chat app (300 MB)
- Total requested: ~8 GB

With demand paging: All run fine. Each loads ~20-30% of itself. Total RAM used: ~3 GB. Smooth.

Without demand paging: Each program must load 100% to run.

Program	Required RAM	Available?
---------	--------------	------------

Browser	1 GB	Yes
Game	4 GB	Yes
IDE	2 GB	Yes
Music player	500 MB	Yes
Chat app	300 MB	NO — total would be 7.8 GB, but each needs contiguous space, fragmentation kills it

Result: You can run maybe 3 programs. Opening a 4th fails with “out of memory.” The system feels primitive.

Thrashing: The Death Spiral

Thrashing: A system state where the CPU spends more time handling page faults (swapping pages in and out) than executing useful instructions.

How thrashing happens — step by step:

Step	What Happens
1	Too many programs running, total working sets > RAM
2	Program A faults, needs a page. RAM full. Evict Program B's page.
3	Program B soon faults for the evicted page. Reload it. Evict Program C's page.
4	Program C soon faults. Reload it. Evict Program A's page.
5	Infinite loop: Every program constantly faulting, constantly reloading, no real work done

Symptoms:

- Disk LED solid on (100% activity)
- CPU usage shows high, but nothing productive happening
- Mouse moves in jerks, typing has seconds of lag
- System effectively frozen

The root cause: Working set size of all processes combined exceeds RAM. No replacement algorithm can fix this — it's a physical impossibility.

The only fix: Kill some programs. Reduce total working set demand.

Step 5: Complete Structured Notes

Core Purpose of Demand Paging

Purpose	How It Achieves It
Creates illusion of large memory	Virtual address space >> physical RAM
Reduces startup time	Load only initial working set
Optimizes RAM usage	Keep only hot data in RAM
Enables multitasking	Many programs share limited RAM
Allows large programs	Program size independent of RAM size

Key Concepts Master List

Concept	One-Sentence Definition
Virtual Memory	Abstraction giving each process a private, large address space independent of physical RAM
Working Set	Pages a process actively uses in a recent time window
Temporal Locality	Recently accessed data is likely to be accessed again soon
Spatial Locality	Nearby data is likely to be accessed soon
Page Fault	Event when CPU accesses a page not in RAM
Thrashing	System state of excessive page faulting causing near-zero useful work
Prefetching	Loading predicted-future pages before they're requested
Hot Data	Frequently accessed pages kept in RAM
Cold Data	Rarely accessed pages left on disk

Important Statements (Write These Down)

1. **Programs do not need all their data at once** — they need only their current working set.
 2. **Memory usage is dynamic** — what is in RAM changes constantly based on what the program is doing.
 3. **Efficiency comes from selective loading** — loading everything is wasteful; loading only what's needed is optimal.
 4. **Demand paging depends on locality** — without predictable access patterns, the system collapses into thrashing.
 5. **Virtual memory is the promise; demand paging is the mechanism that fulfills it.**
-

Step 6: Practice Problems (Full Solutions with All Steps)

Problem 1

Why can a 100 GB program run on a 16 GB RAM system?

Complete Solution:

Step 1 — Identify the contradiction:

- Program size (100 GB) > RAM size (16 GB)
- Direct loading is mathematically impossible
- Therefore, the system must NOT load everything

Step 2 — Explain the mechanism:

- The OS uses **Virtual Memory** to give the program a 100 GB address space
- The program believes it has 100 GB of memory available
- The OS uses **Demand Paging** to load only accessed pages into physical RAM

Step 3 — Quantify the working set:

- At any moment, the program uses only a small subset: the **Working Set**
- Typical working set for a game: 500 MB – 2 GB of active assets
- This fits comfortably in 16 GB RAM

Step 4 — Explain the dynamic process:

- As gameplay progresses, new areas load (page faults → disk reads)
- Old areas are evicted (removed from RAM, written to disk if dirty)
- The working set shifts, but always stays small relative to total program size

Step 5 — Conclusion:

- 100 GB program runs because only ~1-2 GB is ever in RAM at once
 - The other 98-99 GB stays on disk, ready to be paged in on demand
 - Virtual memory provides the illusion; demand paging makes it practical
-

Problem 2

What happens if a program has no locality of reference?

Complete Solution:

Step 1 — Define the scenario:

- Locality of reference = predictable access patterns (temporal + spatial)
- No locality = completely random memory access
- Every access could be to any page, with no relation to previous accesses

Step 2 — Analyze page fault behavior:

- With random access, the working set concept breaks down
- There is no “small set of active pages” — all pages are equally likely
- After loading any page, the next access is probably to a different page
- That different page is probably not in RAM

Step 3 — Calculate the consequence:

- Assume program has 1000 pages, RAM holds 10 pages
- Random access → probability next page is in RAM: $10/1000 = 1\%$
- **Page fault rate: ~99%**
- Each fault = ~10 ms disk access
- For 1,000,000 accesses: $990,000 \text{ faults} \times 10 \text{ ms} = \mathbf{9,900,000 \text{ ms} = 2.75 \text{ hours}}$
- Same accesses with perfect locality: maybe 10 faults total = 100 ms

Step 4 — Identify system state:

- This is **thrashing**
- CPU spends 99.99% of time waiting for disk
- System appears completely frozen
- No replacement algorithm can help — the working set simply exceeds RAM

Step 5 — Real-world parallel:

- This is why iterating arrays sequentially is fast, but jumping around with pointers is slow
 - It's why databases use B-trees (locality) instead of random hash structures for range queries
 - It's why game engines load levels as contiguous chunks, not random assets
-

Problem 3

Why does demand paging improve startup time?

Complete Solution:

Step 1 — Define “startup time”:

- Time from user action (double-click) to program responsiveness (window appears, accepts input)

Step 2 — Old method (no demand paging):

- Entire program must be read from disk into RAM
- Disk read time = (Program Size) / (Disk Bandwidth)
- Example: 500 MB program on HDD with 100 MB/s read speed = 5 seconds minimum
- Plus overhead: OS processing, linking, initialization
- **Total startup: 5-10 seconds**

Step 3 — Demand paging method:

- OS creates process structure, sets up page table (all “Not Present”)
- Loads **only the first page** containing the entry point (~4 KB)
- Disk read time for first page: ~0.04 ms
- CPU begins execution immediately
- **Window can appear after ~20-50 ms**

Step 4 — What happens during “running”:

- User sees window, thinks program is ready
- Program continues loading additional pages on demand
- Most of those pages aren't needed immediately (settings, plugins, background services)
- They load lazily over the next seconds/minutes
- User perceives instant startup

Step 5 — Quantify the savings:

Phase	No Demand Paging	With Demand Paging
Initial disk read	500 MB = 5,000 ms	4 KB = 0.04 ms
Time to first instruction	5,000 ms	0.04 ms
Time to responsive window	5,200 ms	50 ms
Perceived speedup	—	100× faster

Step 6 — Important caveat:

- The program is NOT fully loaded at 50 ms
- If user immediately does something unusual (opens a rare feature), that feature's pages fault in
- There might be a brief pause (100-200 ms) while those pages load
- But 99% of users do normal things first, and those pages are already loaded

Final Intuition (Locked In)

Demand Paging works because programs are predictable. They don't use everything at once. They use small, localized chunks of data over time. The system exploits this predictability to keep only the active chunk in fast memory, while the rest waits cheaply on disk.

Analogy	Reality
You don't wear your entire wardrobe every day	System doesn't load entire program every session
You keep today's clothes in your closet (accessible)	System keeps working set in RAM
Seasonal clothes stay in storage	Cold data stays on disk
When winter comes, you swap	When new level loads, pages swap

3. Deep Theory — How the Hardware Actually Works

Step 0: The Big Picture (No Magic, Just Protocol)

Three components have a strict contract. None of them “knows” the whole story — each does its specific job and passes control.

Component	Physical Location	Exact Job
CPU	Silicon chip on motherboard	Executes machine instructions, generates virtual addresses
MMU	Either on CPU chip (modern) or separate chip (old systems)	Translates virtual → physical, checks permissions, triggers faults
OS	Software in RAM (loaded at boot)	Handles faults, manages disk, decides replacement, updates tables

The rule: The CPU never touches RAM directly with a virtual address. Ever. Every memory access goes through the MMU first.

3.1 The Valid/Invalid Bit — The Gatekeeper

Where This Bit Lives

Inside every **Page Table Entry (PTE)**. The Page Table is an array in RAM. Each entry is typically 4–8 bytes.

Structure of a typical 32-bit PTE:

Frame Number (20 bits)	Which physical frame holds this page
------------------------	--------------------------------------

Valid Bit (1 bit)	1 = in RAM, 0 = not in RAM
Dirty Bit (1 bit)	1 = modified, 0 = clean
Reference Bit (1 bit)	1 = accessed recently (for replacement)
Protection Bits (3 bits)	Read/Write/Execute permissions
Cache Disable Bit (1 bit)	Bypass CPU cache for this page
... (remaining bits)	

Example Page Table:

Virtual Page	Frame Number	Valid	Dirty	Protection
0	5	1	0	Read/Write
1	—	0	—	—
2	12	1	1	Read/Write
3	—	0	—	—
4	8	1	0	Read-Only

What “Invalid” Actually Means (Critical Distinction)

Invalid (Valid = 0) does NOT automatically mean “illegal.”

Scenario	Valid Bit	What It Means	OS Action on Fault
Page on disk, legal	0	Valid page, just not loaded	Load from disk, continue
Page doesn't exist	0	Program exceeded its memory	Kill program (segfault)
Page protected	0 or special	Read-only page written to	Kill or handle error

How the OS tells the difference:

When a page fault occurs, the OS checks:

1. **Is the virtual address within the process's allocated address space?**
 - Look at the process's memory region map (VMAs in Linux)
 - If address < start or > end of any region **ILLEGAL kill**

2. Is the access type allowed?

- Writing to a read-only region? **ILLEGAL** **kill**

3. Is the page mapped to a file or swap space?

- If yes **LEGAL** **load from backing store**

4. Is it a demand-zero page?

- Newly allocated but untouched memory **LEGAL** **allocate fresh zeroed page**

The Library Analogy (Precise Mapping)

Analogy Element	Real System
Book on your desk	Page in RAM (Valid = 1)
Book in library storage	Page on disk (Valid = 0, but legal)
Book that doesn't exist	Invalid address (Valid = 0, illegal)
Librarian (fetches books)	OS Page Fault Handler
Desk size limit	RAM capacity
Rules about which books you can touch	Protection bits

3.2 Full Page Fault Sequence (Every Single Step)

We will trace one complete memory access that causes a fault. No steps skipped.

Step 1: CPU Generates Virtual Address

The instruction:

```
LOAD R1, [0x00005078] // Load 4 bytes from address 0x00005078 into register R1
```

CPU internal operation:

1. Instruction decoder recognizes **LOAD** with memory operand
2. Address generation unit computes effective address: **0x00005078**
3. This is a **virtual address** — the CPU has no idea where it physically lives

Splitting the address (32-bit system, 4 KB pages):

Virtual Address: 0x00005078

Binary: 0000 0000 0000 0000 0101 0000 0111 1000

Page size = 4 KB = 4096 bytes = 2^{12}

Therefore: lower 12 bits = offset, upper 20 bits = page number

Page Number = 0x00005 = 5

Offset = 0x078 = 120

The CPU sends to MMU: “I need Virtual Page 5, Offset 120”

Step 2: MMU Checks Page Table

MMU hardware operation:

1. Receives virtual address from CPU
2. Extracts Page Number (5) and Offset (120)
3. Computes PTE address: $\text{Page Table Base} + (\text{Page Number} \times \text{PTE Size})$
 - If Page Table Base = 0x10000000 and PTE size = 4 bytes
 - PTE address = $0x10000000 + (5 \times 4) = 0x10000014$
4. MMU reads PTE from RAM at 0x10000014

PTE content read by MMU:

Field	Value
Frame Number	— (don't care, invalid)
Valid Bit	0
Dirty Bit	—
Protection	—

Step 3: Valid Bit Check — Two Paths

Path A: Valid = 1 (Page Hit — Not Our Case, But Shown for Contrast)

Action	Detail
--------	--------

MMU extracts Frame Number	Say Frame 23
MMU computes Physical Address	$(23 \times 4096) + 120 = 94208 + 120 = 94328 = 0x00017078$
MMU places address on address bus	0x00017078
RAM responds with data	4 bytes from physical address 0x00017078
Total time	~100 nanoseconds
OS involved?	NO — entirely hardware

Path B: Valid = 0 (Page Fault — Our Case)

Action	Detail
MMU sees Valid = 0	Page not in RAM
MMU triggers Page Fault Exception	Special hardware signal
CPU operation interrupted	Current instruction cannot complete

Step 4: Hardware Trap — The Handoff to OS

What is a Trap?

A trap is a synchronous, intentional CPU exception that transfers control from user mode to kernel mode to request OS services.

“Synchronous” = caused by the current instruction (not random external event)

“Intentional” = the hardware is designed to do this, not a bug

Exact hardware sequence:

Sub-step	What Happens	Time
4.1	CPU finishes current micro-operations that don't touch memory	~1 ns
4.2	CPU saves Program Counter (PC) to stack — points to the faulting instruction	~1 ns

4.3	CPU saves Processor Status Register (flags, mode bits)	~1 ns
4.4	CPU saves General Purpose Registers (or pushes them)	~5 ns
4.5	CPU switches Mode Bit from User (1) to Kernel (0)	~1 ns
4.6	CPU loads new PC from Interrupt Vector Table	~1 ns
4.7	New PC points to OS Page Fault Handler	—
4.8	CPU begins executing OS code	—

Critical detail: The saved PC points to the **faulting instruction itself**, not the next one. This is deliberate — we need to retry this instruction later.

Why the program cannot ignore this:

- The CPU is in kernel mode now — user program has zero control
- The fault happened in hardware, before the instruction completed
- The program doesn't even know it was interrupted (transparent)

Step 5: OS Validates the Access

The OS Page Fault Handler runs. This is C/assembly code in the kernel.

Validation checks (in order):

Check	How OS Does It	Pass?
5.1: Is virtual address in any allocated region?	Check process's VMA (Virtual Memory Area) list	If no segfault
5.2: Is access type allowed?	Check protection bits of the VMA	If no segfault
5.3: Is page mapped to a file?	Check page cache / backing store info	If yes proceed to load
5.4: Is it a copy-on-write page?	Check special COW bit	If yes allocate new page, copy

5.5: Is it a demand-zero page?	Check if newly allocated but unbacked	If yes allocate zeroed page
--------------------------------	---------------------------------------	-----------------------------

Example outcomes:

Scenario	OS Action
Address 0x00005078 is in heap region, write allowed, page on disk	VALID continue to load
Address 0xFFFFFFFF — no region covers it	INVALID send SIGSEGV, terminate
Address in read-only code segment, program tried to write	INVALID send SIGSEGV, terminate

Step 6: Locate Page on Disk

For a valid but non-resident page:

The OS must find where this page lives on disk.

Page Type	Where It Lives	How OS Finds It
Memory-mapped file	The actual file on filesystem	File inode + offset
Swap page (anonymous memory)	Swap partition or swap file	Swap slot number from page table extension
Shared memory	Shared memory backing file	Shared memory object

The “Backing Store”:

Backing Store is the disk-resident storage that holds pages when they are not in RAM. It includes swap space for anonymous memory and the filesystem for memory-mapped files.

OS action:

1. Determine backing store location from PTE extended info or VMA
2. Build an I/O request: “Read disk block X into RAM frame Y”
3. Submit to disk subsystem

Step 7: Find Free Frame in RAM

Case 7A: Free Frame Available

Action	Detail
OS scans frame table / free list	Finds unused physical frame
Say Frame 45 is free	Mark as allocated to this process
Proceed to Step 8	Load page into Frame 45

Case 7B: No Free Frame — Page Replacement Required

Sub-step	Action	Detail
7B.1	Run replacement algorithm	FIFO/LRU/Clock selects victim page
7B.2	Identify victim's process	Say Victim = Page 12 of Process B
7B.3	Check victim's Dirty Bit	Is victim modified?
7B.4a	If Dirty = 0 (Clean)	Just remove from RAM, update victim's PTE to Valid=0
7B.4b	If Dirty = 1 (Dirty)	Must write back to disk first!
7B.5	For dirty victim	Queue disk write: "Write Frame 23 to disk block Y"
7B.6	Wait for write completion	~5-10 ms
7B.7	Victim's PTE updated	Valid=0, Frame cleared
7B.8	Frame now free	Mark as allocated to requesting process

Dirty bit check is critical: Skipping it would lose data modifications.

Step 8: Load Page from Disk

The I/O operation:

Detail	Value
Source	Disk block (identified in Step 6)
Destination	Free frame (identified in Step 7)
Size	4 KB (one page)
Operation	DMA (Direct Memory Access) — disk controller writes directly to RAM without CPU involvement

Timeline:

Phase	Time	What CPU Is Doing
OS submits I/O request	~1 microsecond	Setting up DMA
Disk seek (HDD)	~5 ms	Waiting — context switches to other process
Disk rotational latency	~4 ms	Waiting
Data transfer	~0.5 ms	Waiting — DMA handles it
Interrupt completion	~1 microsecond	OS wakes up, verifies completion

Total: ~10 ms

During this time:

- The faulted process is **blocked** (cannot run)
- CPU runs **other processes** (if any are ready)
- If no other processes CPU idle, waiting for disk

Step 9: Update Page Table

OS actions after disk read completes:

Action	Detail
9.1	Verify DMA completed successfully (check status)

9.2	Write PTE for requested page:
	• Frame Number = newly allocated frame (e.g., 45)
	• Valid Bit = 1
	• Dirty Bit = 0 (just loaded, unmodified)
	• Reference Bit = 1 (just accessed)
	• Protection = copied from VMA permissions
9.3	Update frame table: mark frame 45 as occupied by Page 5 of Process A

Updated Page Table:

Virtual Page	Frame	Valid	Dirty
0	5	1	0
1	—	0	—
2	12	1	1
3	—	0	—
4	8	1	0
5	45	1	0

Step 10: Restart the Faulting Instruction

This is the most critical and often misunderstood step.

Why restart?

- The original `LOAD` instruction never completed
- The CPU was interrupted mid-instruction
- The data was not loaded into register R1
- The program state is exactly as it was before the fault

What the OS does:

Action	Detail
--------	--------

10.1	Restore saved registers from stack
10.2	Restore Program Counter = address of <code>LOAD</code> instruction
10.3	Restore Processor Status Register (switch back to User mode)
10.4	Execute <code>IRET</code> (Interrupt Return) instruction

What the CPU does after IRET:

Action	Detail
10.5	Back in user mode, PC points to <code>LOAD R1, [0x00005078]</code>
10.6	CPU re-executes the same instruction
10.7	Virtual address <code>0x00005078</code> generated again
10.8	MMU checks Page Table — Page 5, Valid = 1, Frame = 45
10.9	MMU computes: $(45 \times 4096) + 120 = 184320 + 120 = 184440 = 0x0002D078$
10.10	Physical address placed on bus
10.11	RAM returns data
10.12	Data stored in R1
10.13	Instruction complete

Total time for restart path: ~100 nanoseconds (RAM access)

Complete Flow Diagram (Write This)

CPU
Executes
`LOAD [addr]`

MMU
Splits addr
into Page+
Offset

Check PTE
for Page

Valid = 1?

YES

NO

Form Phys
Address
(Frame+Off)

PAGE FAULT
EXCEPTION

Access RAM
(~100 ns)

CPU saves
state, trap
to kernel

OS Handler
Validates
access

ILLEGAL

LEGAL

SIGSEGV
Kill prog

Find frame
(replace if

needed)

Load from
disk I/O
(~10 ms)

Update PTE:
Valid=1,
Frame set

Restore CPU
state,
restart
instruction

Instruction
re-executes
HIT DONE

Step 4: Hidden Hardware Components (Detailed)

1. MMU (Memory Management Unit)

Physical location: Inside modern CPUs (was separate chip in 1980s–1990s systems)

Exact functions:

Function	How It Works
Address translation	Receives virtual address, outputs physical address
Page table walk	Reads PTEs from RAM to find translations
Permission check	Compares access type (R/W/X) against PTE protection bits
Valid bit check	If 0, triggers fault instead of translation
TLB management	Caches recent translations for speed

The MMU is pure hardware logic — no software involved in the fast path.

2. Page Table

Physical reality: An array in RAM, one entry per virtual page.

Size calculation example:

- 32-bit virtual address space
- 4 KB pages 2^{20} pages = 1,048,576 pages
- 4 bytes per PTE
- **Total page table size: 4 MB per process**

Problem: 4 MB just for page tables! For 100 processes = 400 MB of RAM just for tables!

Solution: Multi-level page tables

Level	What It Does
Level 1 (Page Directory)	1024 entries, each points to a Level 2 table
Level 2 (Page Table)	1024 entries, each is a PTE

Only Level 1 and active Level 2 tables need to be in RAM. Inactive regions have no Level 2 table save memory.

3. TLB (Translation Lookaside Buffer)

TLB is a small, fast hardware cache inside the MMU that stores recent virtual-to-physical translations.

Why it exists:

- Page table is in RAM reading it takes ~100 ns
- But we need the translation BEFORE we can access the actual data
- Without TLB: every memory access requires TWO RAM accesses (one for PTE, one for data)
- With TLB: most translations are instant

TLB characteristics:

Property	Typical Value
Size	32–128 entries
Organization	Fully associative (any entry can hold any page)
Speed	~1–5 ns (part of CPU pipeline)
What it stores	Virtual Page Physical Frame mapping

TLB operation:

Scenario	Action	Speed
TLB Hit	Translation found in TLB use immediately	~1 ns
TLB Miss	Translation not in TLB walk page table in RAM update TLB	~100 ns

TLB must be flushed on context switch (different process has different page table).

Step 5: What Makes This System Powerful

The Core Illusion

The CPU does not know whether data came from RAM or disk.

Evidence:

- After a page fault and restart, the instruction executes identically
- The data in register R1 is the same whether it was a hit or fault
- Timing is different (100 ns vs 10 ms), but **result is identical**
- Programs cannot detect paging except by measuring time

This is transparency: The mechanism is invisible to correct programs. They run unchanged whether on a system with 4 GB RAM or 64 GB RAM.

Step 6: Edge Cases (Critical)

Case 1: Invalid Access (Segfault)

Scenario: Program tries to read address `0x00000000` (null pointer)

Step	Action
CPU	Generates address <code>0x00000000</code>
MMU	Page 0, Offset 0
MMU	Checks PTE for Page 0
OS Handler	Checks VMA list — no region covers address 0
OS	Access is illegal
OS	Sends SIGSEGV (signal 11) to process
Default action	Terminate process, dump core

Result: Program dies. This is protection — prevents corruption of other processes or the kernel.

Case 2: Thrashing

Scenario: System has 4 GB RAM. Three programs each have 2 GB working sets.

State	Total Working Set	RAM	Result
Running A + B	4 GB	4 GB	Fits exactly, no faults
Add C	6 GB	4 GB	2 GB must be on disk
A faults, needs page	Evict B's page	—	B will fault soon
B faults, needs page	Evict C's page	—	C will fault soon
C faults, needs page	Evict A's page	—	Infinite loop

Symptoms:

- Disk activity 100%, CPU utilization high but progress zero
 - System responsiveness dies
 - Only cure: kill a process or add RAM
-

Step 7: Notebook Summary

Definitions (Formal)

Term	Definition
Valid Bit	Flag in PTE indicating whether page is currently in RAM (1) or not (0)
Page Fault	Hardware exception triggered when CPU accesses a page with Valid=0
Trap	Synchronous CPU exception transferring control to OS kernel
Frame	A page-sized slot in physical RAM
Page Table Entry (PTE)	Record containing frame number, valid bit, dirty bit, protection bits
Backing Store	Disk area holding pages not in RAM
TLB	Hardware cache of recent virtual-to-physical translations
MMU	Hardware unit performing address translation and protection checks

Core Idea (One Sentence)

*Hardware detects the missing page instantly OS resolves the problem correctly
CPU retries the instruction smoothly Program never knows anything happened.*

Step 8: Practice Problems (Full Solutions with All Reasoning)

Problem 1

What triggers a page fault?

Complete Solution:

Step 1 — Identify the exact condition:

A page fault is triggered when the CPU attempts to access a virtual memory address whose corresponding Page Table Entry has the **Valid Bit set to 0**.

Step 2 — Trace the hardware path:

1. CPU generates virtual address
2. MMU extracts page number
3. MMU reads PTE from page table in RAM
4. MMU checks Valid Bit field
5. If Valid Bit = 0 MMU raises Page Fault exception
6. If Valid Bit = 1 MMU proceeds with translation

Step 3 — Distinguish from other faults:

- **Not** a protection fault (those check the Protection bits, not Valid bit)
- **Not** an alignment fault (those check address divisibility)
- Specifically: the page exists in the virtual address space but is not resident in RAM

Step 4 — Valid=0 has two meanings:

- Page is on disk (legal, will be loaded)
- Page is illegal (no backing store, will segfault)

The OS determines which in Step 5 of the fault handler.

Final Answer: A page fault is triggered when the MMU checks the Page Table Entry for a requested virtual page and finds the Valid Bit equal to 0, indicating the page is not currently present in physical RAM.

Problem 2

Why does the CPU restart the instruction after a page fault?

Complete Solution:

Step 1 — Understand what happened during the fault:

- The CPU was executing an instruction like `LOAD R1, [0x00005078]`
- The MMU detected Valid=0 and triggered a trap
- The CPU interrupted the instruction **before it completed**

- Register R1 was **NOT** loaded with data
- The program state is incomplete

Step 2 — Why not just “continue”?

- There is nothing to “continue” — the instruction failed
- The CPU cannot skip to the next instruction because R1 is undefined
- Program logic depends on R1 having the correct value

Step 3 — What the restart achieves:

- After OS loads the page and updates PTE (Valid=1)
- The same instruction is re-executed from the beginning
- This time, MMU finds Valid=1 translation succeeds
- Data is loaded into R1 correctly
- Program continues as if the fault never happened

Step 4 — Why the saved PC points to the faulting instruction:

- The hardware trap mechanism saves the address of the interrupted instruction
- Not the next instruction (that would skip it)
- The OS interrupt return restores this exact PC
- Result: instruction runs again

Step 5 — Analogy:

- You ask for a book. Librarian says “it’s in storage.”
- You wait while they fetch it.
- They hand it to you.
- **You still need to read it.** The original request wasn’t fulfilled — it was delayed.

Final Answer: The CPU restarts the instruction because the original execution was interrupted before completion — the data was never loaded. After the OS brings the page into RAM, the instruction must be re-executed from the beginning to properly complete and produce the correct result.

Problem 3

Why is disk access expensive?

Complete Solution:

Step 1 — Compare raw numbers:

Storage	Access Time	Relative Speed
CPU Register	~0.3 ns	1× (baseline)
L1 Cache	~1 ns	3×
L2 Cache	~4 ns	13×
RAM	~100 ns	333×
SSD (flash)	~100,000 ns (0.1 ms)	333,333×
HDD (mechanical)	~10,000,000 ns (10 ms)	33,333,333×

Step 2 — HDD mechanics (why so slow):

Component	Action	Time
Read/write head	Must move to correct track	~5 ms (seek)
Platter	Must rotate to correct sector	~4 ms (latency)
Data transfer	Read 4 KB as platter spins	~0.5 ms
Total		~9.5 ms

Physical movement of metal parts = milliseconds. Electronic RAM = nanoseconds. The gap is physics.

Step 3 — SSD mechanics (faster but still slow):

Factor	Impact
Flash memory read	~25 microseconds (fast)
Controller overhead	~50 microseconds
Protocol latency (SATA/NVMe)	~10–100 microseconds
Total typical	~100 microseconds

SSD is ~100× faster than HDD, but still ~1000× slower than RAM.

Step 4 — Why this matters for paging:

Metric	Value
--------	-------

One page fault (HDD)	~10 ms
One RAM access	~100 ns
Ratio	100,000× slower
CPU instructions in 10 ms	~30,000,000 instructions (at 3 GHz)
CPU instructions in 100 ns	~300 instructions

One disk access = CPU could have executed 30 million instructions.

Step 5 — System-level impact:

- During a page fault, the faulted process is blocked
- CPU may switch to other work (if available)
- But if all processes are faulting CPU idle, disk saturated
- System throughput collapses

Final Answer: Disk access is expensive because it involves physical mechanical movement (HDD) or complex electronic protocols (SSD), making it 100,000 to 1,000,000 times slower than RAM. In the time required for one disk access, a modern CPU could execute tens of millions of instructions, making page faults a massive performance penalty.

Final Intuition (Locked In)

Demand Paging is a three-party cooperation system with strict roles:

Party	Responsibility	Speed
Hardware (MMU)	Detect problems instantly, enforce rules	Nanoseconds
OS (Kernel)	Solve problems correctly, manage resources	Microseconds to milliseconds
CPU + Program	Continue execution smoothly, remain ignorant	Nanoseconds (when working)

The beauty: Each layer does what it's best at. Hardware is fast but dumb — it detects. OS is smart but slower — it decides. CPU is fast and focused — it executes. Together they create the illusion of infinite, fast memory.

4. Mathematics: The Cost of a Page Fault

Step 0: What Are We Actually Measuring?

Every time your program accesses memory, one of two things happens:

Scenario	What Happens	Time
Hit	The page is already in RAM	Fast (~100–200 ns)
Miss (Page Fault)	The page is not in RAM must load from disk	Very slow (~5–10 ms)

The question: If I run my program for a long time, what is the *average* time per memory access?

This average is called: Effective Access Time (EAT)

Effective Access Time (EAT) is the statistical average time required for a memory access operation, accounting for both the common fast path (RAM hit) and the rare slow path (page fault to disk).

Think of it like calculating your average commute time:

- Usually: 20 minutes (clear roads)
- Sometimes: 90 minutes (bad traffic)
- Average = weighted by how often each happens

Step 1: Understanding the Real Components

Given Typical Values

Component	Time	In Nanoseconds (for calculation)
-----------	------	----------------------------------

RAM access	~100–200 nanoseconds	200 ns
Page Fault (full handling)	~5–10 milliseconds	8,000,000 ns

Why page faults are “millions of nanoseconds”:

Sub-operation	Approximate Time
MMU detects fault, CPU trap	~50 ns
OS handler runs, validates access	~500 ns
Find free frame / run replacement	~1–5 microseconds
Disk seek + rotation + read (HDD)	~8–10 milliseconds
Update page table, restart instruction	~100 ns
Total	~8,000,000 ns (8 ms)

The gap: $8,000,000 \div 200 = 40,000\times$ slower

Step 2: The Core Formula (Built from Scratch)

Where This Formula Comes From

This is a **weighted average** — a standard probability formula.

Scenario: You flip a weighted coin:

- With probability **(1–p)**: you take the fast path (RAM)
- With probability **p**: you take the slow path (Page Fault)

Expected (average) time:

$$\begin{aligned} \text{EAT} = & (\text{Probability of fast path}) \times (\text{Time of fast path}) \\ & + (\text{Probability of slow path}) \times (\text{Time of slow path}) \end{aligned}$$

The Formula

$$EAT = (1 - p) \cdot T_{RAM} + p \cdot T_{PF}$$

Symbol	Meaning	Example Value
p	Probability of a page fault (fault rate)	0.001 = 0.1%
$(1 - p)$	Probability of a normal RAM access (hit rate)	0.999 = 99.9%
T_{RAM}	Time for one RAM access	200 ns
T_{PF}	Time to handle one complete page fault	8,000,000 ns

Intuition check: If $p = 0$ (never fault), $EAT = T_{RAM} = 200$ ns. Perfect.

If $p = 1$ (always fault), $EAT = T_{PF} = 8,000,000$ ns. Disaster.

Step 3: Fully Worked Example (Every Step Shown)

Given:

- $T_{RAM} = 200$ ns
 - $T_{PF} = 8,000,000$ ns
 - $p = 0.001$ (0.1% of accesses cause a fault)
-

Step 3.1: Calculate the RAM Hit Contribution

$$(1 - p) = 1 - 0.001 = 0.999$$

$$(1 - p) \times T_{RAM} = 0.999 \times 200$$

Multiplication shown:

```

200
× 0.999
-----
200 × 1 = 200.000
- 200 × 0.001 = 0.200
-----
199.800

```

Or directly:

$$0.999 \times 200 = (1 - 0.001) \times 200 = 200 - 0.2 = 199.8 \text{ ns}$$

Result: The “fast path” contributes **199.8 ns** to the average.

Step 3.2: Calculate the Page Fault Contribution

$$p \times T_{PF} = 0.001 \times 8,000,000$$

Multiplication shown:

$$0.001 = 1/1000$$

$$\text{So: } 8,000,000 \div 1000 = 8,000$$

Or:

$$0.001 \times 8,000,000 = \frac{8,000,000}{1000} = 8,000 \text{ ns}$$

Result: The “slow path” contributes **8,000 ns** to the average.

Step 3.3: Add Both Contributions

$$EAT = 199.8 + 8,000$$

Addition shown:

$$\begin{array}{r} 199.8 \\ + 8,000.0 \\ \hline 8,199.8 \end{array}$$

$$EAT = 8,199.8 \text{ ns} \approx 8,200 \text{ ns}$$

Final Answer

$EAT \approx 8,200 \text{ ns}$

Step 4: Interpretation (The Part That Matters)

Compare the Numbers

Metric	Value
Pure RAM time	200 ns
EAT with 0.1% faults	8,200 ns
Slowdown factor	$8,200 \div 200 = 41\times$

What This Means

A **0.1% fault rate** — one fault per thousand accesses — makes your memory system **41 times slower** than it should be.

Visual breakdown of where the 8,200 ns comes from:

Total EAT: 8,200 ns

From RAM hits (99.9% of accesses): 199.8 ns (2.4%)

From page faults (0.1% of accesses): 8,000 ns

97.6% of your average access time comes from 0.1% of your accesses.

This is the definition of an **outlier dominating the average**.

Step 5: Why This Happens (The Deep Reason)

The Mathematical Reason

The formula is:

$$\text{EAT} = (1 - p) \cdot T_{\text{RAM}} + p \cdot T_{\text{PF}}$$

When $T_{\text{PF}} \gg T_{\text{RAM}}$ (page fault time is MUCH larger than RAM time), the second term dominates even for tiny p .

General principle:

Small probability × Huge cost = Large impact

Analogy	Real System
---------	-------------

One expensive lawsuit per 1000 customers	One page fault per 1000 accesses
999 customers cost \$10 each = \$9,990	999 hits at 200 ns = 199,800 ns
1 lawsuit costs \$1,000,000	1 fault at 8,000,000 ns = 8,000,000 ns
Average per customer: ~\$1,010	Average per access: ~8,200 ns
99% of average cost from 0.1% of events	Same principle

Step 6: What Must Be True for Good Performance?

The Goal

We want EAT to be very close to T_{RAM} .

$$\text{EAT} \approx T_{\text{RAM}}$$

This only happens when the page fault term is negligible:

$$p \cdot T_{\text{PF}} \approx 0$$

Which requires:

$$p \approx 0$$

Conclusion: Page fault rate must be kept **extremely close to zero** for the memory system to perform well.

Step 7: Reverse Calculation — Finding Acceptable Fault Rate

The Problem

You are a system designer. You want $\text{EAT} \leq 220 \text{ ns}$ (only 10% worse than pure RAM).

Given:

- $T_{\text{RAM}} = 200 \text{ ns}$
- $T_{\text{PF}} = 8,000,000 \text{ ns}$
- Desired EAT = 220 ns

- Find: acceptable fault rate p
-

Step 7.1: Write the Equation

$$220 = (1 - p) \times 200 + p \times 8,000,000$$

Step 7.2: Expand the Right Side

$$220 = 1 \times 200 - p \times 200 + p \times 8,000,000$$

$$220 = 200 - 200p + 8,000,000p$$

Step 7.3: Collect Like Terms

Group the p terms:

$$220 = 200 + (-200p + 8,000,000p)$$

$$220 = 200 + (8,000,000 - 200)p$$

$$220 = 200 + 7,999,800p$$

Step 7.4: Isolate p

Subtract 200 from both sides:

$$220 - 200 = 7,999,800p$$

$$20 = 7,999,800p$$

Step 7.5: Solve for p

$$p = \frac{20}{7,999,800}$$

Division shown:

$$p = \frac{20}{7,999,800} = \frac{1}{399,990} \approx 0.0000025$$

Or:

$$p \approx 2.5 \times 10^{-6}$$

Step 7.6: Convert to Percentage

$$p \approx 0.0000025 = 0.00025\%$$

Final Result

$$p \approx 0.00025\%$$

Meaning: Out of every **400,000 memory accesses**, you can afford **only 1 page fault** if you want to stay within 10% of RAM speed.

The Insight

Fault Rate	EAT	Slowdown
0%	200 ns	1× (baseline)
0.00025%	220 ns	1.1× (acceptable)
0.001%	8200 ns	41× (unacceptable)
0.01%	~80,200 ns	~400× (catastrophic)

The acceptable fault rate is 4,000× smaller than the “tiny” 0.1% rate.

This is why systems fight so hard to minimize faults — TLBs, good replacement algorithms, prefetching, huge pages, etc.

Step 8: Real System Connection

What Systems Actually Do

Technique	How It Helps	Effect on EAT
TLB	Caches translations, avoids page table walks	Reduces T_{RAM} slightly (100 ns 1 ns for translation)

Good Replacement (LRU)	Keeps likely-to-be-used pages in RAM	Reduces p
Working Set Model	Ensures each process gets its active pages	Prevents p from spiking
Prefetching	Loads pages before they're needed	Converts faults to hits
Huge Pages	Reduces TLB misses, fewer page table entries	Faster translation, fewer overheads

All of these target p (fault rate) first, T_{RAM} second.

Step 9: Complete Notebook Summary

Formula (Write This Exactly)

$$\text{EAT} = (1 - p) \cdot T_{\text{RAM}} + p \cdot T_{\text{PF}}$$

Key Insights

#	Insight
1	EAT is a weighted average of fast and slow paths
2	Page fault time is ~40,000× RAM time
3	Even tiny fault rates (0.1%) cause massive slowdowns (41×)
4	The fault term dominates EAT when $p > 0.00025\%$
5	System design priority: minimize p at all costs

The One-Sentence Takeaway

Memory system performance is determined not by how fast RAM is, but by how rarely you must leave RAM.

Step 10: Practice Problems (Full Step-by-Step Solutions)

Problem 1

Given: $T_{\text{RAM}} = 100 \text{ ns}$, $PF = 5,000,000 \text{ ns}$, $p = 0.002$

Find: EAT

Complete Solution:

Step 1 — Identify values:

- $T_{\text{RAM}} = 100 \text{ ns}$
- $T_{\text{PF}} = 5,000,000 \text{ ns}$
- $p = 0.002$
- $(1 - p) = 1 - 0.002 = 0.998$

Step 2 — Write the formula:

$$EAT = (1 - p) \cdot T_{\text{RAM}} + p \cdot T_{\text{PF}}$$

Step 3 — Calculate RAM hit contribution:

$$(1 - p) \cdot T_{\text{RAM}} = 0.998 \times 100$$

$$0.998 \times 100 = (1 - 0.002) \times 100 = 100 - 0.2 = 99.8 \text{ ns}$$

Step 4 — Calculate page fault contribution:

$$p \cdot T_{\text{PF}} = 0.002 \times 5,000,000$$

$$0.002 = \frac{2}{1000}$$

$$\frac{2}{1000} \times 5,000,000 = \frac{10,000,000}{1000} = 10,000 \text{ ns}$$

Step 5 — Add contributions:

$$EAT = 99.8 + 10,000 = 10,099.8 \text{ ns}$$

Step 6 — Round and interpret:

$EAT \approx 10,100 \text{ ns}$

Interpretation:

- Pure RAM: 100 ns
- With 0.2% faults: 10,100 ns
- **Slowdown: 101×**
- The fault term (10,000 ns) is 99% of the total.

Problem 2

Why is reducing page faults more important than optimizing RAM speed?

Complete Solution:

Step 1 — State the EAT formula:

$$EAT = (1 - p) \cdot T_{RAM} + p \cdot T_{PF}$$

Step 2 — Analyze what we can improve:

Approach	What Changes	Typical Improvement
Faster RAM	Reduce T_{RAM}	200 ns → 100 ns = 2× faster
Fewer faults	Reduce p	0.001 → 0.000001 = 1000× fewer faults

Step 3 — Calculate impact of faster RAM (without changing p):

Original:

$$EAT_{old} = 0.999 \times 200 + 0.001 \times 8,000,000 = 199.8 + 8,000 = 8,199.8 \text{ ns}$$

With 2× faster RAM ($T_{RAM} = 100 \text{ ns}$):

$$EAT_{fast \text{ ram}} = 0.999 \times 100 + 0.001 \times 8,000,000 = 99.9 + 8,000 = 8,099.9 \text{ ns}$$

Improvement: 8,199.8 → 8,099.9 = **1.2% faster** (barely noticeable)

Step 4 — Calculate impact of fewer faults (without changing RAM):

Reduce p from 0.001 to 0.000001:

$$EAT_{low \text{ faults}} = 0.999999 \times 200 + 0.000001 \times 8,000,000$$

$$= 199.9998 + 8 = 207.9998 \text{ ns}$$

Improvement: 8,199.8 → 208 = **39× faster** (massive)

Step 5 — Compare:

Strategy	EAT	Speedup
Do nothing	8,200 ns	1×
2× faster RAM only	8,100 ns	1.01×
1000× fewer faults only	208 ns	39×

Step 6 — Explain mathematically:

The fault term $p \cdot T_{PF}$ dominates EAT when p is not extremely small.

- RAM term: $(1 - p) \cdot T_{RAM} \approx 200$ ns (small, bounded)
- Fault term: $p \cdot T_{PF}$ — if $p = 0.001$, this is 8,000 ns (40× larger)

Improving the small term by 2× changes total by ~1%.

Improving the large term by 1000× changes total by ~99%.

Step 7 — Real-world analogy:

You have a commute:

- 999 days: 10-minute drive (highway)
- 1 day: 8-hour flight (emergency)

Average = $(999 \times 10 + 1 \times 480) / 1000 = 10.47$ minutes

Option A: Buy a faster car, drive takes 5 minutes

New average = $(999 \times 5 + 1 \times 480) / 1000 = 5.48$ minutes

Improvement: 2× (only on good days)

Option B: Avoid the emergency flight entirely

New average = 10 minutes

Improvement: Eliminates 99.5% of delay

Conclusion: The rare catastrophic event dominates the average. Eliminating it is vastly more impactful than optimizing the common case.

Final Answer: Reducing page faults is more important because the fault term ($p \cdot T_{PF}$) dominates EAT by orders of magnitude. Even modest reductions in fault rate yield massive speedups, while doubling RAM speed barely moves the needle when faults are present.

Final Intuition (Locked In)

Memory system performance is NOT about the speed of RAM. It is entirely about avoiding the slow path — the page fault. A memory system with slow RAM and zero faults is infinitely faster than a memory system with fast RAM and frequent faults.

System	RAM Speed	Fault Rate	EAT	Verdict
A	200 ns	0%	200 ns	Perfect
B	100 ns	0.1%	~8,100 ns	40× worse than A
C	500 ns	0%	500 ns	Still great

The fault rate is the only knob that matters.

5. Page Replacement Strategies

(Complete Understanding)

Step 0: The Core Problem

The situation:

- 1. A page fault occurs
- 2. The needed page is on disk
- 3. RAM is completely full — no free frames exist
- 4. **Question:** Which page currently in RAM should we throw out to make space?

Why this decision is critical:

Bad Choice	Good Choice
Evict a page you'll need in 2 seconds	Evict a page you won't need for 5 minutes
That page faults back in immediately	That page stays evicted, no fault
Wastes disk I/O, wastes time	Saves disk I/O, saves time
EAT increases	EAT decreases

The algorithm that makes this choice is called:

A Page Replacement Algorithm is a strategy used by the OS to select which resident page in RAM should be removed (evicted) to accommodate a newly requested page from disk.

5.1 FIFO (First-In, First-Out)

Core Logic

Evict the page that has been in RAM the longest time — the first one that entered.

Data structure: A **Queue** (First-In-First-Out)

Queue: [Oldest] P1 P2 P3 [Newest]

Evict this one (front of queue)

When a new page loads, it joins the back of the queue.
When eviction needed, remove from the front.

Complete Worked Example

Given:

- RAM capacity: 3 frames
- Reference string (page access order): A B C D A B

We track: RAM contents, Queue order, Fault/Hit, Eviction decision

Access 1: Page A

Action	Detail
RAM before	[empty]
Is A in RAM?	No PAGE FAULT
RAM full?	No (0/3 frames used)
Action	Load A into Frame 0
Queue	[A]
RAM after	[A]

Fault count: 1

Access 2: Page B

Action	Detail
RAM before	[A]
Is B in RAM?	No PAGE FAULT

RAM full?	No (1/3 frames used)
Action	Load B into Frame 1
Queue	[A, B]
RAM after	[A, B]

Fault count: 2

Access 3: Page C

Action	Detail
RAM before	[A, B]
Is C in RAM?	No PAGE FAULT
RAM full?	No (2/3 frames used)
Action	Load C into Frame 2
Queue	[A, B, C]
RAM after	[A, B, C]

Fault count: 3

Access 4: Page D

Action	Detail
RAM before	[A, B, C]
Is D in RAM?	No PAGE FAULT
RAM full?	YES (3/3 frames used)
Eviction needed?	YES
FIFO choice	Queue front = A (oldest)
Action	Evict A from Frame 0, Load D into Frame 0
Queue	[B, C, D] (A removed from front, D added to back)

RAM after	[B, C, D]
-----------	------------------

Fault count: 4

Access 5: Page A

Action	Detail
RAM before	[B, C, D]
Is A in RAM?	No (we evicted it!) PAGE FAULT
RAM full?	YES
FIFO choice	Queue front = B (oldest now)
Action	Evict B, Load A
Queue	[C, D, A]
RAM after	[C, D, A]

Fault count: 5

Access 6: Page B

Action	Detail
RAM before	[C, D, A]
Is B in RAM?	No (we evicted it!) PAGE FAULT
RAM full?	YES
FIFO choice	Queue front = C
Action	Evict C, Load B
Queue	[D, A, B]
RAM after	[D, A, B]

Fault count: 6

Final Summary Table

Access	Page	RAM Before	Evict	RAM After	Fault?
1	A	[]	—	[A]	
2	B	[A]	—	[A, B]	
3	C	[A, B]	—	[A, B, C]	
4	D	[A, B, C]	A	[B, C, D]	
5	A	[B, C, D]	B	[C, D, A]	
6	B	[C, D, A]	C	[D, A, B]	

Total Faults: 6 out of 6 accesses (0 hits)

Why so bad? FIFO evicted A at step 4, but A was needed at step 5. Then evicted B, but B was needed at step 6. FIFO had no idea these were coming.

Belady's Anomaly (The Shocking Problem)

Belady's Anomaly: For some page reference strings, increasing the number of available frames (more RAM) actually increases the number of page faults when using FIFO.

This violates intuition: normally more RAM = better. FIFO breaks this.

Proving Belady's Anomaly with a Concrete Example

Reference string: 1 2 3 4 1 2 5 1 2 3 4 5

Case A: 3 Frames

Step	Access	RAM	Action	Fault?
1	1	[1]	Load	
2	2	[1, 2]	Load	
3	3	[1, 2, 3]	Load	
4	4	[2, 3, 4]	Evict 1 (oldest), Load 4	

5	1	[3, 4, 1]	Evict 2, Load 1	
6	2	[4, 1, 2]	Evict 3, Load 2	
7	5	[1, 2, 5]	Evict 4, Load 5	
8	1	[1, 2, 5]	Already in RAM	HIT
9	2	[1, 2, 5]	Already in RAM	HIT
10	3	[2, 5, 3]	Evict 1, Load 3	
11	4	[5, 3, 4]	Evict 2, Load 4	
12	5	[5, 3, 4]	Already in RAM	HIT

Total faults with 3 frames: 9

Case B: 4 Frames

Step	Access	RAM	Action	Fault?
1	1	[1]	Load	
2	2	[1, 2]	Load	
3	3	[1, 2, 3]	Load	
4	4	[1, 2, 3, 4]	Load	
5	1	[1, 2, 3, 4]	Hit	
6	2	[1, 2, 3, 4]	Hit	
7	5	[2, 3, 4, 5]	Evict 1, Load 5	
8	1	[3, 4, 5, 1]	Evict 2, Load 1	
9	2	[4, 5, 1, 2]	Evict 3, Load 2	
10	3	[5, 1, 2, 3]	Evict 4, Load 3	
11	4	[1, 2, 3, 4]	Evict 5, Load 4	
12	5	[2, 3, 4, 5]	Evict 1, Load 5	

Total faults with 4 frames: 10

The Anomaly Proven

Frames	Faults
3	9
4	10 WORSE!

More RAM caused MORE faults. This is Belady's Anomaly.

Why it happens with FIFO:

- With 3 frames, page 1 was evicted early and didn't cause much trouble
- With 4 frames, page 1 stayed longer, then when evicted at step 7, it was needed again at step 8 — fault
- The "extra" frame held pages that weren't useful, displacing pages that were about to be needed

FIFO ignores usage patterns — it only cares about arrival time, not whether a page will be needed soon.

5.2 Optimal Algorithm (The Theoretical Ideal)

Core Logic

Evict the page that will not be used for the longest period of time in the future.

This requires **perfect future knowledge** — you look ahead in the reference string and see which current page is needed furthest in the future (or never again).

Complete Worked Example

Given:

- RAM capacity: 3 frames
- Reference string: A B C A B D

We simulate with future-knowledge:

Access 1: A

Action	Detail
RAM	[A]
Fault?	

Access 2: B

Action	Detail
RAM	[A, B]
Fault?	

Access 3: C

Action	Detail
RAM	[A, B, C]
Fault?	

Access 4: A

Action	Detail
RAM	[A, B, C]
A is in RAM	HIT

Access 5: B

Action	Detail
RAM	[A, B, C]
B is in RAM	HIT

Access 6: D (Fault — RAM full)

Future lookup: From step 6, look ahead in reference string:

Page	Next Use	Distance
A	Step 7 (if exists) or never	Let's say never in this example
B	Step 8 (if exists) or never	Let's say never
C	Step 9 (if exists) or never	Let's say never

If all never used again: Evict any (say C).

If A used at step 10, B at step 20, C never: Evict **C** (furthest = infinity = never).

Action	Detail
RAM before	[A, B, C]
Optimal choice	Evict C (never used again, or furthest)
RAM after	[A, B, D]
Fault?	

Why Optimal Is The Best Possible

Proof sketch: At any eviction point, choosing the page with the furthest next use minimizes the chance of an immediate re-fault. Any other choice would evict a page that is needed sooner, guaranteeing at least as many faults.

But: It requires knowing the future reference string. In a real running system, the OS cannot predict what the program will access next.

Usage: Optimal is used only as a **benchmark** — to measure how close real algorithms (FIFO, LRU, Clock) get to the theoretical best.

5.3 LRU (Least Recently Used) — The Gold Standard

Core Logic

Evict the page that has not been accessed for the longest time.

Key assumption: If you haven't used a page recently, you probably won't use it soon. (Temporal locality — past behavior predicts future behavior.)

Complete Worked Example

Given:

- RAM capacity: 3 frames
- Reference string: A B C A D B

We track: RAM contents + **usage timestamps** (or order of last use)

Access 1: A

Action	Detail
RAM	[A]
Last used order	A = time 1
Fault?	

Access 2: B

Action	Detail
RAM	[A, B]
Last used order	A = time 1, B = time 2
Fault?	

Access 3: C

Action	Detail
RAM	[A, B, C]
Last used order	A=1, B=2, C=3
Fault?	

Access 4: A

Action	Detail
RAM	[A, B, C]

A is in RAM update timestamp	
Last used order	B=2, C=3, A=4 (A is now most recent)
Fault?	HIT

Access 5: D (Fault — RAM full)

Determine LRU: Which page was used longest ago?

Page	Last Used At
A	Step 4
B	Step 2 LONGEST AGO
C	Step 3

LRU choice: Evict **B**

Action	Detail
RAM before	[A, B, C]
Evict	B (least recently used)
Load	D
RAM after	[A, C, D]
Last used order	A=4, C=3, D=5
Fault?	

Access 6: B

Action	Detail
RAM before	[A, C, D]
Is B in RAM?	No (we evicted it) PAGE FAULT
Determine LRU	A=4, C=3, D=5 C is oldest
Evict	C

Load	B
RAM after	[A, D, B]
Fault?	

Final Summary

Access	Page	RAM Before	Evict (LRU)	RAM After	Fault?
1	A	[]	—	[A]	
2	B	[A]	—	[A, B]	
3	C	[A, B]	—	[A, B, C]	
4	A	[A, B, C]	— (hit, update)	[A, B, C]	
5	D	[A, B, C]	B	[A, C, D]	
6	B	[A, C, D]	C	[A, D, B]	

Total faults: 5 out of 6

Compare to FIFO on same string: FIFO would have evicted A at step 5 (oldest arrival), then faulted on A if accessed next. LRU correctly kept A because it was recently used.

Why Perfect LRU Is Expensive to Implement

Problem: To know which page is “least recently used,” you must track the exact time of every single memory access.

Method 1: Counter Method

Hardware required:

- A global counter register (64-bit, increments on every memory access)
- Each PTE stores the counter value at its last access

On every memory access:

1. CPU accesses address MMU translates
2. MMU increments global counter: `counter = counter + 1`
3. MMU writes counter value into PTE’s “last used” field

4. To evict: scan all PTEs, find minimum counter value

Cost:

- Every access requires a counter increment + PTE write = **slow**
- Eviction requires scanning ALL pages = **$O(n)$** where n = number of pages

Verdict: Too expensive for hardware.

Method 2: Stack Method (Doubly-Linked List)

Structure: Maintain a doubly-linked list of pages in usage order.

Most Recently Used [D] [A] [C] [B] Least Recently Used

On access to page X:

1. If X in list: remove it from current position, move to front (MRU end)
2. If X not in list (fault): add to front, evict from back if full

Cost:

- Every access requires pointer updates (4 pointer writes) = **moderate**
- Eviction is always from back = **$O(1)$**
- But needs hardware support for linked list manipulation = **complex**

Verdict: Better than counters, but still complex for hardware MMUs.

5.4 Clock Algorithm (Second Chance) — The Practical Solution

Why It Exists

LRU is too expensive to implement perfectly. Clock approximates LRU behavior using only a single bit per page and a circular pointer — extremely cheap hardware.

The Structure

Physical layout:

P1 Clock Hand points here
R=1

P4 P2
R=0 R=1

P3
R=0

- Pages arranged in a **circular list**
 - Each page has a **Reference Bit** ®: 1 = used recently, 0 = not used recently
 - **Clock Hand**: A pointer that moves around the circle
-

The Algorithm (Step-by-Step)

When a page fault occurs and eviction is needed:

Step	Action
1	Look at page where clock hand points
2	If R = 1: This page was used recently give it a “second chance”
	• Set R = 0 (remove the chance)
	• Move clock hand to next page
	• Go to Step 1 (check next page)
3	If R = 0: This page was NOT used recently evict it
	• Replace this page with the new page

	• Set new page's R = 1
	• Move clock hand to next page
	• Done

Complete Worked Example

Given:

- RAM capacity: 4 frames
- Pages in circle: P1, P2, P3, P4
- Initial state: All R bits shown below
- Clock hand starts at P1
- Reference string: NewPage X (needs to be loaded)

Initial State:

Page	R Bit	Position
P1	1	Clock hand here
P2	1	
P3	0	
P4	1	

Step-by-Step Execution

Round 1: Hand at P1, R=1

Action	Detail
Check P1	R = 1
Decision	Give second chance
Set P1.R	0
Move hand	P2

State after Round 1:

- P1: R=0
 - P2: R=1 hand here
 - P3: R=0
 - P4: R=1
-

Round 2: Hand at P2, R=1

Action	Detail
Check P2	R = 1
Decision	Give second chance
Set P2.R	0
Move hand	P3

State after Round 2:

- P1: R=0
 - P2: R=0
 - P3: R=0 hand here
 - P4: R=1
-

Round 3: Hand at P3, R=0

Action	Detail
Check P3	R = 0
Decision	EVICT
Action	Remove P3, load X in its place
Set X.R	1
Move hand	P4

Final State:

- P1: R=0
 - P2: R=0
 - X: R=1 (replaced P3)
 - P4: R=1 hand here
-

Why Clock Works (The Intuition)

Scenario	Clock Behavior
Page accessed frequently	Its R bit keeps getting set to 1 never evicted
Page accessed once then idle	R=1 gets one second chance R=0 next scan, evicted
Page never accessed	R stays 0 evicted on first encounter

Clock approximates LRU:

- Recently used pages survive (like LRU)
 - But uses only 1 bit instead of timestamps or linked lists
 - Hardware cost: minimal
-

Step 6: Algorithm Comparison Table

Algorithm	Core Idea	Fault Rate	Implementation Cost	Real Usage
FIFO	Evict oldest arrival	Poor (Belady's anomaly)	Very cheap (queue)	Rarely used alone
Optimal	Evict furthest future use	Best possible	Impossible (needs future)	Benchmark only
LRU	Evict least recently used	Very good	Expensive (timestamps/list)	Approximated, not pure
Clock	Evict unreferenced (approx LRU)	Good	Very cheap (1 bit + pointer)	Widely used in real OS

Step 7: What Makes a Good Algorithm? (Design Principles)

A good page replacement algorithm must balance three factors:

Factor	Why It Matters
Low fault rate	Directly improves EAT and system performance
Exploits locality	Matches how real programs actually behave
Cheap to implement	Complex algorithms slow down every memory access

Modern OS reality:

- Pure LRU is too expensive
- FIFO is too dumb
- **Clock (and variants like Enhanced Clock, WSClock) hit the sweet spot**

Step 8: The Stack Property (Hidden Concept)

The Stack Property (also called the inclusion property): For a given reference string, the set of pages in memory for a smaller number of frames is always a subset of the set of pages in memory for a larger number of frames.

Algorithms with Stack Property:

- LRU
- Optimal

Result: More frames → never more faults (monotonic improvement)

Algorithms WITHOUT Stack Property:

- FIFO

Result: More frames CAN increase faults → Belady's Anomaly

Why this matters: The Stack Property guarantees predictable behavior. FIFO's unpredictability makes it unsuitable for systems where adding RAM should always help.

Step 9: Practice Problems (Full Solutions)

Problem 1

Which algorithm is best theoretically?

Complete Solution:

Step 1 — Identify the theoretical standard:

The “best” algorithm is defined as the one that produces the minimum possible number of page faults for any given reference string.

Step 2 — Name the algorithm:

Optimal Page Replacement Algorithm (also called MIN or Belady’s Optimal)

Step 3 — Explain its strategy:

At each eviction point, it removes the page whose next use is furthest in the future (or never used again).

Step 4 — Prove why it’s optimal:

- Any other choice would evict a page that is needed sooner
- This guarantees at least as many future faults
- Therefore, Optimal’s choice minimizes total faults

Step 5 — State why it’s not used in practice:

- Requires knowledge of the entire future reference string
- In a running system, the OS cannot predict future memory accesses
- Impossible to implement in real hardware

Final Answer: The **Optimal Algorithm** is theoretically best because it evicts the page with the furthest next use, guaranteeing the minimum possible page faults. However, it requires future knowledge and is impossible to implement in real systems, serving only as a performance benchmark.

Problem 2

Why is LRU better than FIFO?

Complete Solution:

Step 1 — Compare core strategies:

Algorithm	Eviction Based On
FIFO	Time of arrival into RAM
LRU	Time of last actual use by CPU

Step 2 — Analyze with an example:

Reference string: A B C A D

With 3 frames:

FIFO behavior:

- Step 4: A is accessed (hit)
- Step 5: D faults, RAM full [A, B, C]
- FIFO evicts A (arrived first)
- But A was JUST used at step 4 will probably be used again soon
- **Bad decision**

LRU behavior:

- Step 4: A accessed (hit), update timestamp
- Step 5: D faults, RAM full [A, B, C]
- LRU checks last use: A=step4, B=step2, C=step3
- LRU evicts B (least recent)
- A stays because it was recently used
- **Good decision**

Step 3 — Explain the fundamental reason:

Principle	FIFO	LRU
Uses locality?	No	Yes
Adapts to program behavior?	No	Yes
Predicts future needs?	No	Yes (via past pattern)

Step 4 — Quantify the impact:

Programs exhibit **temporal locality** — recently used pages are likely to be used again.

- FIFO ignores this, treating a just-used page the same as an idle page
- LRU exploits this, protecting recently used pages
- Result: LRU typically has 20–40% fewer faults than FIFO on real workloads

Step 5 — Note the trade-off:

Aspect	FIFO	LRU
Implementation	Very cheap	Expensive
Fault rate	Higher	Lower

LRU is better but costs more. In practice, approximations like Clock are used.

Final Answer: LRU is better than FIFO because it exploits **temporal locality** — the principle that recently used pages are likely to be reused. FIFO blindly evicts the oldest arrival regardless of usage, often evicting actively needed pages. LRU protects recently used pages, leading to significantly fewer page faults and better EAT.

Problem 3

Why is Clock used instead of LRU?

Complete Solution:

Step 1 — State the problem with pure LRU:

Perfect LRU requires tracking the exact time of every memory access.

Implementation costs:

Method	Hardware Needed	Per-Access Cost	Eviction Cost
Counter	64-bit counter + timestamp per PTE	Increment counter + write timestamp	Scan all pages for min
Stack	Doubly-linked list in hardware	4 pointer updates per access	O(1) from back, but complex

Both methods add significant overhead to **every single memory access**, slowing down the fast path.

Step 2 — Explain Clock’s advantages:

Feature	Clock Implementation
Hardware per page	1 bit (Reference bit)
Hardware global	1 pointer (clock hand)
Per-access cost	Zero — just set R=1 on access (can be done by MMU in parallel)
Eviction cost	Simple circular scan, usually finds victim quickly

Step 3 — Compare behavior:

Scenario	LRU	Clock
----------	-----	-------

Frequently used page	Protected (recent timestamp)	Protected (R stays 1)
Once-used page	Evicted when oldest	Gets one second chance, then evicted
Idle page	Evicted immediately	Evicted when hand reaches it with R=0

Clock is a **close approximation** of LRU with **fraction of the cost**.

Step 4 — Real-world usage:

OS	Algorithm Used
Linux	Clock-like (LRU approximation with active/inactive lists)
Windows	Working Set algorithm (Clock variant)
Unix variants	Clock or Enhanced Clock

Step 5 — The trade-off summary:

Factor	Pure LRU	Clock
Fault rate	Excellent	Good (slightly worse than LRU)
Implementation cost	High	Very low
Hardware complexity	Complex	Simple
Real-world usage	Rare	Universal

Final Answer: Clock is used instead of pure LRU because it provides **similar performance** (good fault reduction via approximating LRU behavior) at **minimal hardware cost** (only 1 bit per page + 1 pointer). Pure LRU requires expensive timestamp tracking or complex linked-list hardware on every memory access. The slight increase in faults is vastly outweighed by the simplicity and speed of the Clock implementation.

Final Intuition (Locked In)

Page Replacement is about making smart eviction decisions under uncertainty:

Algorithm	Analogy	Intelligence Level
FIFO	Throw out the book you've had the longest, regardless of whether you're reading it	Blind — no usage awareness
Optimal	Magically know which book you won't need until next year	Omniscient — impossible
LRU	Throw out the book you haven't opened in the longest time	Smart — uses past behavior
Clock	Give each book one "stay" vote; if you haven't touched it since last check, it goes	Practical smart — cheap approximation

The real world chooses Clock because intelligence must be affordable. A slightly dumber decision made instantly beats a perfect decision that takes too long to compute.

6. Visualizing the Process (LRU — Deep Walkthrough)

Step 0: What We Are Simulating

We are going to trace through a **reference string** (a sequence of page requests) and watch exactly what happens inside RAM frame by frame, access by access.

Given:

- **Reference sequence:** 1 2 3 4 2
- **RAM size:** 3 frames
- **Algorithm:** LRU (Least Recently Used)

Our representation rule:

[Leftmost ... Rightmost]

Oldest/LRU Newest/MRU

- **Left** = Least Recently Used (candidate for eviction)
- **Right** = Most Recently Used (protected)
- **On a hit:** The accessed page moves to the rightmost position (newest)
- **On a miss (RAM not full):** New page added to rightmost
- **On a miss (RAM full):** Evict leftmost, add new page to rightmost

Step-by-Step Execution (Every Micro-Action Tracked)

Step 1: Request = 1

Check: Is 1 in RAM?

RAM: []

RAM is empty. 1 is not present.

Result: MISS (Page Fault)

Action: Load 1 into RAM. Since it's the only page, it's both oldest and newest.

RAM: [1]

Only page = MRU = LRU

Fault count: 1

Step 2: Request = 2

Check: Is 2 in RAM?

RAM: [1]

2 $\neq$ 1. Not present.

Result: MISS (Page Fault)

Action: Load 2. RAM not full (1/3 used). Add to right.

RAM: [1, 2]

LRU MRU

Interpretation: 1 was used at step 1, 2 was used at step 2. So 1 is "older" (less recent) than 2.

Fault count: 2

Step 3: Request = 3

Check: Is 3 in RAM?

RAM: [1, 2]

$3 \neq 1$, $3 \neq 2$. Not present.

Result: MISS (Page Fault)

Action: Load 3. RAM not full (2/3 used). Add to right.

RAM: [1, 2, 3]

LRU MRU

Interpretation:

- 1 used at step 1 oldest
- 2 used at step 2 middle
- 3 used at step 3 newest

RAM is now FULL (3/3). Next miss will require eviction.

Fault count: 3

Step 4: Request = 4

Check: Is 4 in RAM?

RAM: [1, 2, 3]

$4 \neq 1$, $4 \neq 2$, $4 \neq 3$. Not present.

Result: MISS (Page Fault)

RAM status: FULL (3/3) **EVICTION REQUIRED**

LRU Decision Process:

Page	Last Used At	Age Rank
1	Step 1	Oldest EVICT THIS
2	Step 2	Middle

3	Step 3	Newest
---	--------	--------

Eviction: Remove 1 (leftmost, LRU)

Before eviction:

RAM: [1, 2, 3]

EVICT

After eviction (1 removed):

RAM: [2, 3]

LRU MRU

Load 4: Add to rightmost (MRU position)

RAM: [2, 3, 4]

LRU MRU

Interpretation:

- 2 used at step 2 now oldest
- 3 used at step 3 middle
- 4 used at step 4 newest

Fault count: 4

Step 5: Request = 2

Check: Is 2 in RAM?

RAM: [2, 3, 4]

2 = 2. **YES, PRESENT!**

Result: HIT

CRITICAL ACTION — REORDERING:

This is the most important and most misunderstood part of LRU. A hit is **not passive**. The accessed page becomes the **most recently used** and must move to the rightmost position.

Before reorder:

RAM: [2, 3, 4]

LRU MRU

Step 5a: Remove 2 from its current position (leftmost):

RAM: [_, 3, 4]

LRU MRU

Step 5b: Add 2 to the rightmost (MRU) position:

RAM: [3, 4, 2]

LRU MRU

Final state after hit:

RAM: [3, 4, 2]

Interpretation of new order:

- 3 used at step 3 now oldest
- 4 used at step 4 middle
- 2 used at step 5 (just now) newest

Fault count: 4 (no change — it's a hit)

Complete Result Summary Table

Step	Request	RAM Before	Check	Result	Action Taken	RAM After (LRU MRU)	Faults
1	1	[]	Not in RAM	Miss	Load 1	[1]	1
2	2	[1]	Not in RAM	Miss	Load 2	[1, 2]	2
3	3	[1, 2]	Not in RAM	Miss	Load 3	[1, 2, 3]	3
4	4	[1, 2, 3]	Not in RAM	Miss	Evict 1, Load 4	[2, 3, 4]	4
5	2	[2, 3, 4]	In RAM	Hit	Move 2 to MRU	[3, 4, 2]	4

Final: 4 faults, 1 hit out of 5 accesses

Step 6: Deep Insight — What Actually Happened

Observation 1: LRU Tracks Usage, Not Arrival

Step	Page	Last Use
1	1	Step 1
2	2	Step 2
3	3	Step 3
4	4	Step 4
5	2	Step 5 updated!

At step 4, page 2 was the oldest (used at step 2). But at step 5, page 2 was just used, so it became newest. **The “age” of a page changes dynamically with every access.**

Observation 2: Hits Cause Reordering (Critical)

If we did NOT reorder on hit, what would happen?

WRONG implementation (no reorder on hit):

Step	Request	RAM (wrong)	Result
5	2	[2, 3, 4]	Hit
Next: 1		[2, 3, 4]	1 not present, evict 2?

But 2 was JUST used! Evicting it would be stupid. The correct implementation moves 2 to MRU, protecting it.

Correct implementation (reorder on hit):

Step	Request	RAM (correct)	Result
5	2	[3, 4, 2]	2 is now protected (MRU)
Next: 1		[3, 4, 2]	1 not present, evict 3 (LRU)

3 hasn't been used since step 3. Correctly evicted.

Observation 3: FIFO Would Have Made a Worse Decision

Same sequence with FIFO (for contrast):

FIFO tracks arrival order, not usage:

Step	Request	FIFO RAM (arrival order)	Action
1	1	[1]	Load
2	2	[1, 2]	Load
3	3	[1, 2, 3]	Load
4	4	[2, 3, 4]	Evict 1 (arrived first)
5	2	[2, 3, 4]	Hit (no reorder in FIFO)

In this specific short sequence, FIFO gets lucky — 2 is still there. But if the sequence continued 1 :

Step	Request	FIFO RAM	Action
6	1	[3, 4, 1]	Evict 2 (arrived first)

FIFO evicts 2, but 2 was just used at step 5! **Blind decision.**

LRU on 1 after step 5:

Step	Request	LRU RAM	Action
6	1	[4, 2, 1] or evict 3	3 is LRU, evict 3

LRU correctly protects 2 (recently used) and evicts 3 (idle since step 4).

Step 7: LRU vs FIFO Side-by-Side Comparison

Extended sequence: 1 2 3 4 2 1

FIFO Trace:

Step	Request	RAM (arrival order)	Result	Evict
1	1	[1]	Miss	—
2	2	[1, 2]	Miss	—
3	3	[1, 2, 3]	Miss	—
4	4	[2, 3, 4]	Miss	1
5	2	[2, 3, 4]	Hit	—
6	1	[3, 4, 1]	Miss	2

FIFO faults at step 6 (evicts 2, which was just used at step 5).

LRU Trace:

Step	Request	RAM (usage order)	Result	Evict
1	1	[1]	Miss	—
2	2	[1, 2]	Miss	—
3	3	[1, 2, 3]	Miss	—
4	4	[2, 3, 4]	Miss	1

5	2	[3, 4, 2]	Hit	— (reorder)
6	1	[4, 2, 1]	Miss	3

LRU faults at step 6 but evicts 3 (not used since step 4), protecting 2 (used at step 5).

Score after 6 steps:

- FIFO: 5 faults
- LRU: 5 faults (same here, but LRU made smarter choices — difference grows with longer sequences)

Step 8: The Stack Property (Why LRU Behaves Well)

LRU maintains a stack-like ordering of pages by recency of use.

What this means:

At any moment, the RAM contents ordered from LRU to MRU form a “stack” where:

- The bottom (LRU) is the page to evict
- The top (MRU) is the most protected page
- Any page’s position changes dynamically based on access

Why this avoids Belady’s Anomaly:

With the Stack Property, if you increase frames from N to $N+1$:

- The N pages in the smaller RAM are always a subset of the $N+1$ pages in larger RAM
- You only ADD protection, never remove it
- More frames → equal or fewer faults (monotonic)

FIFO lacks this because arrival order doesn’t correlate with usefulness.

Step 9: Common Mistakes (With Concrete Counter-Examples)

Mistake 1: “Hit Means Nothing Changes”

Wrong thinking: “2 is already in RAM, so RAM stays [2, 3, 4]”

Why this breaks LRU:

If RAM stays [2, 3, 4] after hitting 2:

- Next request: 1 (miss)
- Evict leftmost = 2
- But 2 was JUST used! It should be protected.

Correct: Move 2 to MRU [3, 4, 2]

- Next request: 1 (miss)
- Evict leftmost = 3 (not used since step 4)
- 2 is protected at MRU position

Rule: Every access (hit or miss that loads) updates the recency order.

Mistake 2: Confusing “Oldest Arrival” with “Least Recently Used”

Concept	What It Tracks	Algorithm
Oldest arrival	When did the page enter RAM?	FIFO
Least recently used	When was the page last accessed by CPU?	LRU

Example where they differ:

Sequence: A B C A D

At step 4 (A is accessed again):

- **FIFO order:** A arrived first, B second, C third [A, B, C] by arrival
- **LRU order:** A used at step 4, B at step 2, C at step 3 [B, C, A] by last use

At step 5 (D faults):

- **FIFO evicts:** A (oldest arrival) — but A was JUST used!
- **LRU evicts:** B (least recent use) — correct, B idle since step 2

Mistake 3: Not Updating Position After Access

Broken implementation trace:

Step	Request	RAM (broken)	Result
------	---------	--------------	--------

1	1	[1]	Miss
2	2	[1, 2]	Miss
3	1	[1, 2]	Hit
4	3	[1, 2, 3]	Miss (wrong: should have reordered 1 to MRU)

What should happen at step 3:

- 1 is hit → move to MRU
- RAM: [2, 1] (2 is now LRU, 1 is MRU)

What broken implementation does:

- 1 is hit → no reorder
- RAM: [1, 2] (1 still appears old)

At step 4 (3 misses):

- Broken: evicts 1 (appears LRU) — but 1 was just used!
- Correct: evicts 2 (true LRU) — 2 hasn't been used since step 2

Result of broken implementation: Behaves like FIFO, loses all LRU advantages.

Step 10: Practice Problem (Full Solution with All Steps)

Problem

Given:

- Sequence: 1 2 1 3 4
- Frames: 3
- Algorithm: LRU

Trace completely:

Step 1: Request = 1

Check	RAM: [] 1 not present
Result	Miss

Action	Load 1
RAM	[1]

Step 2: Request = 2

Check	RAM: [1] 2 not present
Result	Miss
Action	Load 2
RAM	[1, 2]

Step 3: Request = 1

Check	RAM: [1, 2] 1 is present!
Result	HIT
Action	Move 1 to MRU position
Before	[1, 2]
Remove 1	[, 2]
Add 1 to right	[2, 1]

Critical: 1 moves from leftmost to rightmost. 2 becomes LRU.

Step 4: Request = 3

Check	RAM: [2, 1] 3 not present
Result	Miss
RAM full?	No (2/3 used)
Action	Load 3 to MRU position
RAM	[2, 1, 3]

Order: 2 (step 2), 1 (step 3), 3 (step 4)

Step 5: Request = 4

Check	RAM: [2, 1, 3] 4 not present
Result	Miss
RAM full?	Yes (3/3)
Eviction needed?	Yes

LRU Decision:

Page	Last Used At	Position
2	Step 2	Leftmost = LRU EVICT
1	Step 3	Middle
3	Step 4	Rightmost = MRU

Evict 2:

Before: [2, 1, 3]

After removal: [_, 1, 3]

Load 4 to MRU: [1, 3, 4]

Final Answer Table

Step	Request	RAM Before	Result	Action	RAM After (LRU MRU)
1	1	[]	Miss	Load	[1]
2	2	[1]	Miss	Load	[1, 2]
3	1	[1, 2]	Hit	Reorder: move 1 to MRU	[2, 1]
4	3	[2, 1]	Miss	Load	[2, 1, 3]
5	4	[2, 1, 3]	Miss	Evict 2, Load 4	[1, 3, 4]

Total: 4 faults, 1 hit

Why This Is Correct (Verification)

At step 5, we evicted 2. Let's verify:

- 2 was last used at step 2
- 1 was last used at step 3
- 3 was last used at step 4

2 is indeed the least recently used. Correct eviction.

If we had NOT reordered at step 3 (Mistake 3), RAM would be [1, 2, 3] at step 5, and we'd evict 1 — but 1 was used at step 3, more recent than 2's step 2. Wrong!

Step 11: Final Intuition (Locked In)

LRU is not about when a page arrived. It is about when a page was last touched by the CPU. Every single access — hit or load — reshuffles the protection order. The page at the left edge is always the most vulnerable. The page at the right edge is always the safest. This dynamic reordering is what makes LRU intelligent.

Action	Effect on RAM Order
Miss (RAM not full)	Add new page to right (MRU)
Miss (RAM full)	Evict leftmost (LRU), add new page to right
Hit	Move accessed page to right (MRU) — THIS IS THE KEY

Think of it as a popularity contest that updates in real time:

- Just used = most popular = safest
 - Idle = losing popularity = at risk
 - Leftmost = least popular = evicted
-

7. Real-World CPU Behavior (What Actually Happens in Systems)

Step 0: Why This Section Matters

Everything up to now has been theory — formulas, algorithms, idealized sequences. This section answers:

Question	Real-World Answer
Does the CPU actually set the dirty bit?	Yes, in hardware, automatically
Do phones page fault like desktops?	No — they kill apps instead
What does thrashing feel like as a user?	Your computer becomes unusable
How do real OS decide replacement?	Clock + complex heuristics, not pure LRU

7.1 Desktop CPUs (Intel / AMD — x86 Architecture)

The Dirty Bit: Hardware-Level Reality

Dirty Bit = A flag in the Page Table Entry that indicates whether the page has been modified (written to) since it was loaded from disk.

Physical location: Bit 6 of the x86 PTE (Page Table Entry) — hardwired into the CPU's MMU.

How the Dirty Bit Works (Full Hardware Sequence)

Step 1: Page Loaded from Disk

Action	Detail
--------	--------

OS loads page into RAM frame 23	Disk RAM
OS sets PTE	Frame = 23, Valid = 1, Dirty = 0 , Protection = R/W
CPU not involved yet	OS does this during fault handling

PTE after load:

Frame Number: 23 (bits 12-31)
 Dirty Bit: 0 CLEAN
 Accessed Bit: 0
 ...

Step 2: CPU Writes to the Page

Instruction:

ADD [0x00005000], 1 // Memory at virtual address 0x5000 += 1

Hardware sequence:

Sub-step	Who	What Happens
2.1	CPU	Generates virtual address 0x5000
2.2	MMU	Translates to physical frame 23, offset 0x000
2.3	MMU	Checks PTE: Valid=1, Protection=R/W write allowed
2.4	MMU	Automatically sets Dirty Bit = 1 (hardware action)
2.5	MMU	Places physical address on bus
2.6	RAM	Receives write, updates value
2.7	CPU	Continues execution

Critical point: The OS is **not involved** in setting the dirty bit. The MMU hardware does this automatically on every write. The OS only **reads** the dirty bit later during eviction.

Step 3: Page Replacement — The Dirty Bit Decision

Later, the OS needs to evict a page. It checks the dirty bit.

Case A: Dirty Bit = 0 (Clean Page)

Check	Result
Dirty?	No
Disk copy	Identical to RAM copy
Action	Discard immediately — just update PTE to Valid=0
Disk write?	NONE
Time	~1 microsecond

Case B: Dirty Bit = 1 (Dirty Page)

Check	Result
Dirty?	Yes
Disk copy	Out of date — RAM copy is newer
Action	Must write back to disk first
Then	Update PTE to Valid=0, free frame
Time	~5-10 milliseconds (disk I/O)

Why the Dirty Bit Is Essential

Scenario without dirty bits:

Action	Time
Every eviction	Write to disk (just in case)
Even if page unmodified	5-10 ms wasted
System with 100 evictions/second	500-1000 ms/sec spent on unnecessary writes

Scenario with dirty bits:

Page Type	% of Pages	Action	Time
Clean	~60-70%	Discard	~1 μ s
Dirty	~30-40%	Write back	~5-10 ms

Savings: Avoiding unnecessary writes on 60-70% of evictions = massive throughput improvement.

Deep System Insight: The Accessed Bit (Often Ignored)

x86 PTEs also have an **Accessed Bit** (bit 5):

- Set by hardware on **any** access (read or write)
- Used by OS for replacement algorithms (Clock algorithm uses this)
- OS can clear it to track recent usage

Together:

- **Accessed Bit** = “Has this page been touched recently?” (for replacement)
 - **Dirty Bit** = “Has this page been modified?” (for write-back)
-

7.2 Mobile Systems (Android / iOS)

The Mobile Problem

Component	Desktop	Mobile
RAM	8-64 GB	2-12 GB
Storage	HDD/SSD (replaceable)	Flash (soldered, limited lifespan)
Battery	Plugged in or large	Small, precious
Thermal budget	Large heatsinks	Minimal

Critical constraint: Flash memory has **limited write cycles** (typically 1,000–10,000 per cell). Heavy swapping would wear out the storage.

The Mobile Strategy: App Killing Instead of Swapping

When RAM is full on mobile:

Desktop Approach	Mobile Approach
Evict page to swap space on disk	Kill the entire app process
Keep app alive, page back later	App terminates, frees all its RAM
Slow (disk I/O)	Fast (just free memory)
App state preserved	App state lost (must reload)

What You Experience as a User

Scenario	What Happened
You switch back to Instagram, it reloads from start	OS killed it to free RAM
You had a game paused, now it's at the title screen	OS killed it
Spotify keeps playing in background	OS marked it as "active audio" protected from killing

Android specifically:

- Uses **OOM Killer** (Out-Of-Memory Killer) with priority scores
- Apps get "importance" ratings: foreground > service > cached
- Cached apps killed first

iOS specifically:

- More aggressive app killing
- Apps must save state before being killed
- When you return, app restores from saved state (appears seamless)

Key Insight: Different Optimization Goals

System	Optimizes For	Accepts
Desktop	Raw performance	Power consumption, disk wear

Mobile	Battery + hardware longevity	App reloads, state loss
--------	------------------------------	-------------------------

8. Thrashing: System Failure Mode

Formal Definition

Thrashing is a system state where the combined working sets of active processes exceed available RAM, causing the system to spend the majority of CPU time handling page faults rather than executing useful instructions.

The Mathematical Model of Thrashing

Given:

- n = number of active processes
- W = working set size per process (pages)
- F = total frames in RAM

Condition for healthy operation:

$$n \times W \leq F$$

Condition for thrashing:

$$n \times W > F$$

The Thrashing Cycle (Step-by-Step)

Setup: 3 processes, each needs 50 pages working set. RAM has 100 frames.

Check: $3 \times 50 = 150 > 100$ **THRASHING INEVITABLE**

Time	What Happens	Result
T1	Process A faults, needs page X	OS loads X, evicts page from B
T2	Process B soon needs its evicted page	B faults, OS loads it, evicts page from C
T3	Process C soon needs its evicted page	C faults, OS loads it, evicts page from A
T4	Process A needs its evicted page	A faults again...

...	Infinite loop	No process makes progress
-----	----------------------	---------------------------

CPU utilization during thrashing:

Phase	CPU State
Fault handling	Running OS code (useful work, but not user program)
Disk wait	Idle — waiting for disk I/O
Instruction execution	Minimal — program barely runs

Result: CPU shows 100% “busy” but throughput approaches zero.

The User Experience of Thrashing

Symptom	Cause
Disk LED solid on	Constant disk I/O from page faults
Mouse moves in jerks	OS can’t process input promptly
Typing has seconds of lag	Keyboard buffer fills, OS can’t drain
Task Manager shows high CPU	CPU is in OS fault handlers, not apps
Nothing responds to clicks	All processes blocked on faults

The analogy: Not just “switching books on your desk.” It’s:

Every sentence you read requires a new book. You fetch it, read one sentence, someone else needs your desk space, they take your book. You need it again, fetch it again. You spend 100% of your time fetching, 0% reading.

Solution: The Working Set Model

Working Set Model: The OS monitors each process’s working set size and ensures that the sum of all working sets fits in RAM before admitting a process to run.

OS Decision Logic:

Condition	Action
-----------	--------

$\sum \text{working sets} \leq F$	Run all processes normally
$\sum \text{working sets} > F$	Suspend (swap out) one or more processes

What “suspend” means:

- Move entire process to disk (all its pages)
- Free all its frames
- Reduce $\sum \text{working sets}$
- Resume suspended process later when RAM available

The trade-off:

Approach	Result
Keep all processes running	Thrashing — nobody makes progress
Suspend some processes	Fewer processes run, but each runs fast

Key principle: Throughput = $n \times \text{progress per process}$. When thrashing, n is high but progress per process ≈ 0 . Better to reduce n and make progress > 0 .

9. Complete System Pipeline (Your Mental Model)

Here is the full chain, from CPU request to completion:

CPU requests
virtual addr

MMU checks
Page Table

Valid=1 Valid=0

Form	PAGE
phys	FAULT
addr	

RAM	OS loads
access	from disk
~100ns	~10ms

RAM full?

No	Yes
----	-----

Load	Run
into	Replacement
free	Algorithm
frame	(Clock)

Check
Dirty Bit

Dirty=0 Dirty=1

Discard Write to
immediately disk first
~1 μ s ~10ms

Update
Page Table
Valid=1

Restart
instruction

Continue
execution

10. Practice Problem (Deep Analysis)

Given

- Frames = 3
- Sequence: A B C A D
- Algorithm = FIFO

FIFO Trace (Complete)

Step 1: A

Check	RAM: []
Result	Miss
RAM full?	No
Action	Load A
RAM	[A]
Queue	[A]
Faults	1

Step 2: B

Check	RAM: [A]
Result	Miss
RAM full?	No
Action	Load B
RAM	[A, B]
Queue	[A, B]
Faults	2

Step 3: C

Check	RAM: [A, B]
Result	Miss
RAM full?	No
Action	Load C
RAM	[A, B, C]
Queue	[A, B, C]
Faults	3

RAM is now FULL.

Step 4: A

Check	RAM: [A, B, C]
Result	HIT — A is present
FIFO action on hit	None — FIFO does not reorder on hit
RAM	[A, B, C] (unchanged)
Queue	[A, B, C] (unchanged)
Faults	3

Step 5: D

Check	RAM: [A, B, C]
Result	Miss
RAM full?	Yes
FIFO choice	Queue front = A (oldest arrival)
Action	Evict A , Load D
RAM	[B, C, D]
Queue	[B, C, D]
Faults	4

Final FIFO Result

Step	Request	RAM	Result	Faults
1	A	[A]	Miss	1
2	B	[A, B]	Miss	2
3	C	[A, B, C]	Miss	3
4	A	[A, B, C]	Hit	3

5	D	[B, C, D]	Miss (evict A)	4
---	---	-----------	----------------	---

Total Page Faults: 4

What If We Used LRU? (Critical Comparison)

Same sequence, same frames, LRU algorithm:

Step 1: A

RAM	[A]
Order	A=MRU=LRU
Faults	1

Step 2: B

RAM	[A, B]
Order	A (step 1), B (step 2)
Faults	2

Step 3: C

RAM	[A, B, C]
Order	A(1), B(2), C(3)
Faults	3

Step 4: A (HIT — CRITICAL DIFFERENCE)

Check	A present
Result	Hit
LRU Action	Move A to MRU
Before	[A, B, C]
After reorder	[B, C, A]

New order	B(2), C(3), A(4)
Faults	3

A is now protected. B is the LRU candidate.

Step 5: D

Check	D not present
Result	Miss
RAM full?	Yes
LRU choice	B (least recent — used at step 2)
Action	Evict B , Load D
RAM	[C, A, D]
Order	C(3), A(4), D(5)
Faults	4

Side-by-Side Final Comparison

Step	Request	FIFO RAM	LRU RAM	FIFO Result	LRU Result
1	A	[A]	[A]	Miss	Miss
2	B	[A, B]	[A, B]	Miss	Miss
3	C	[A, B, C]	[A, B, C]	Miss	Miss
4	A	[A, B, C]	[B, C, A]	Hit	Hit (reordered)
5	D	[B, C, D]	[C, A, D]	Miss (evict A)	Miss (evict B)

Same fault count (4) for this short sequence, but different evictions.

Why the difference matters:

If the sequence continued **A** :

- **FIFO:** A was evicted at step 5 **fault on A**

- **LRU:** A is protected at MRU **hit on A**

LRU wins on continuation because it protected the recently-used page.

Final Intuition (Complete System Understanding)

The full memory system is a pipeline of decisions, each made by a different layer:

Layer	Decision	Speed
CPU	What address do I need?	Nanoseconds
MMU	Is it in RAM? (Valid bit)	Nanoseconds
Hardware	Set dirty/accessed bits automatically	Nanoseconds
OS (fault path)	Load from disk, where is it?	Milliseconds
OS (replacement)	Which page to evict?	Microseconds
OS (dirty check)	Write back or discard?	Microseconds to milliseconds
CPU (restart)	Retry the instruction	Nanoseconds

The final truth:

Memory system performance is determined by how rarely the slow path executes. A “fast” RAM with frequent faults is slower than a “slow” RAM with perfect locality. The entire architecture — TLB, dirty bits, Clock algorithm, working set monitoring — exists to keep the CPU on the fast path.

Metric	What Actually Matters
RAM speed	Secondary — only relevant on hits
Page fault rate	Primary — determines EAT
Replacement intelligence	Primary — determines future fault rate
Dirty bit efficiency	Primary — reduces disk writes

Working set fit	Primary — prevents thrashing
-----------------	-------------------------------------
